A PROJECT REPORT ON

CYBER THREAT DETECTION BASED ON ARTIFICIAL NEURAL NETWORKS USING EVENT PROFILE

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By

JAVIDI GAYATHRI(22VF1M3345)

Under the Guidance of

MS S.GURU JYOTHI
Asst.Professor
Dept.of CSE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SR INTERNATIONAL INSTITUTE OF TECHNOLOGY
Rampally(V),Keesara(M),Medchal Dist., Hyderabad-501301
(Affiliated To JNTUH,Approved by AICTE NEW DELHI)

HYDERABAD

(2022-2026)

SR INTERNATIONAL INSTITUTE OF TECHNOLOGY

Rampally(V), Keesara(M), Medchal Dist., Hyderabad-501301

(Affiliated to JNTUH, Approved by AICTE NEW DELHI)

Department Of Computer Science & Engineering

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that this project report entitled "**Cyber Threat Detection Based on Artifical Neural Networks Using Event Profiles** "" submitted by **Javidi Gayathri** bearing Roll No: **22VF1M3345** in Partial fulfilment of the requirement for the Award of Degree of Bachelor of Technology in Computer Science & Engineering, is a record of Bonafide work carried out by them.

Dr. PulagamHari Krishna,

Head Of The Department,

Department of Computer Science & Engineering

SRIIT,Hyderabad

SR INTERNATIONAL INSTITUTE OF TECHNOLOGY

Rampally(V),Keesara(M),Medchal Dist.,Hyderabad-501301

(Affiliated To JNTUH,Approved by AICTE NEW DELHI)

Department of computer science & engineering

CERTIFICATE

This is to certify that this project work entitled "**A Cyber Threat Detection Based on Artifical Neural Networks Using Event Profiles"** submitted by **Javidi Gayathri** bearing Roll no:**22VF1M3345**in partial fulfillment of the award of degree of Bachelors of Technology in Computer Science & Engineering is a record of Bonafide work carried out by them under my guidance. The result of investigation enclosed in the report have been verified and found satisfactory. The result embodied in this project report has not been submitted to any other institution or University or Institute for the award of any other degree or diploma.

Signature of
Project Guide
Ms S. Guru Jyothi
Asst. Professor
Dept. of CSE

Signature of HOD
Dr. Pulagam Hari Krishna
Dept. of CSE

SR INTERNATIONAL INSTITUTE OF TECHNOLOGY

Rampally(V), Keesara(M), Medchal Dist., Hyderabad-501301

(Affiliated to JNTUH, Approved by AICTE NEW DELHI)

Department Of Computer Science & Engineering

DECLARATION BY THE CANDIDATES

I, **Javidi Gayathri,** bearing RollNo: **22VF1M3345** hereby certify that the project entitled "**A Cyber Threat Detection Based on Artifical Neural Networks Using Event Profiles**''carried out under the guidance of **Ms S.GURU JYOTHI** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in computer science and engineering is a record of Bonafide work carried out by we. The results embodied in this project report have not been reproduced/copied from any source, and have not been submitted to any other university or institute for the award of any other or diploma.

JAVIDI GAYATHRI(22VF1M3345)

SR INTERNATIONAL INSTITUTE OF TECHNOLOGY

Rampally(V),Keesara(M),Medchal Dist., Hyderabad-501301
(Affiliated To JNTUH,Approved by AICTE NEW DELHI)

Department of Computer Science & Engineering

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success, who guided, inspired and helped us for the completion of our project work. We would like to express our deep indebtedness and whole hearted thanks to **Dr. D. Sravan Kumar,** Principal of SRIIT, for allowing us to undertake this project. We are also thankful for his valuable suggestion and cooperation during the development of this project.

We are also thankful to **Dr. Pulagam Hari Krishan**, Head of the Department of Department of computer science and Engineering, who has continuously helped in critical suggestion and extension of proper working atmosphere, finally evolved into this research work.

We are also thankful to project guide Department of Computer Science and Engineering for his valuable Guidance throughout our project. We are also thankful to all our faculty and the staff members for their support. We are thankful to each and every person who helped us directly or indirectly in doing our work.

JAVIDI GAYATHRI (22VF1M3345)

TABLE OF CONTENTS

Abstract… **1**

Chapter 1

INTRODUCTION **2**

Chapter 2

LITERATURE SURVEY **5**

Chapter 3

SYSTEM ANALYSIS **11**

Chapter 4

FEASIBILITY STUDY **15**

Chapter 5

SYSTEM DESIGN… **19**

Chapter 6

IMPLEMENTATION **32**

Chapter 7

CODE **75**

Chapter 8

TESTING AND VALIDATION **102**

Chapter 9

OUTPUT SCREENS… **110**

CONCLUSION **126**

REFERNCES **131**

List Of Figures

Fig.No	Name of the figures	Page no
4.6	Architecture Diagram	18
5.2.1	Use case Diagram	26
5.2.2	Class Diagram	27
5.2.3	Sequence Diagram	28
5.2.4	Activity Diagram	29

ABSTRACT

One of the major challenges in cyber security is the provision of an automated and effective cyber-threats detection technique. In this paper, we present an AI technique for cyber-threats detection, based on artificial neural networks. The proposed technique converts multitude of collected security events to individual event profiles and use a deep learning-based detection method for enhanced cyber- threat detection. For this work, we developed an AI-SIEM system based on a combination of event profiling for data preprocessing and different artificial neural network methods, including FCNN, CNN, and LSTM. The system focuses on discriminating between true positive and false positive alerts, thus helping security analysts to rapidly respond to cyber threats. All experiments in this study are performed by authors using two benchmark datasets (NSLKDD and CICIDS2017) and two datasets collected in the real world. To evaluate the performance comparison with existing methods, we conducted experiments using the five conventional machine-learning methods (SVM, k-NN, RF, NB, and DT). Consequently, the experimental results of this study ensure that our proposed methods are capable of being employed as learning-based models for network intrusion-detection, and show that although it is employed in the real world, the performance outperforms the conventional machine-learning methods.

CHAPTER 1

INTRODUCTION

With the emergence of artificial intelligence (AI) techniques, learning-based approaches for detecting cyber attacks, have become further improved, and they have achieved significant results in many studies. However, owing to constantly evolving cyber attacks, it is still highly challenging to protect IT systems against threats and malicious behaviors in networks. Because of various network intrusions and malicious activities, effective defenses and security considerations were given high priority for finding reliable solutions .

Traditionally, there are two primary systems for detecting cyber-threats and network intrusions. An intrusion prevention system (IPS) is installed in the enterprise network, and can examine the network protocols and flows with signature-based methods primarily. It generates appropriate intrusion alerts, called the security events, and reports the generating alerts to another system, such as SIEM. The security information and event management (SIEM) has been focusing on collecting and managing the alerts of IPSs. The SIEM is the most common and dependable solution among various security operations solutions to analyze the collected security events and logs . Moreover, security analysts make an effort to investigate suspicious alerts by policies and threshold, and to discover malicious behavior by analyzing correlations among events, using knowledge related to attacks.

A learning-based method geared toward determining whether an attack occurred in a large amount of data can be useful to analysts who need to instantly analyze numerous events. According to, information security solutions generally fall into two categories: analyst-driven and machine learning-driven solutions. Analyst- driven solutions rely on rules determined by security experts called analysts.

Meanwhile, machine learning-driven solutions used to detect rare or anomalous patterns can improve detection of new cyber threats .Nevertheless, while learning-based approaches are useful in detecting cyber attacks in system networks, we observed that existing learning-based approaches have four main limitations.

First, learning-based detection methods require labeled data, which enable the training of the model and evaluation of generated learning models. Furthermore, it is not straightforward to obtain such labeled data at a scale that allow accurate training of a model. Despite the need for labeled data, many commercial SIEM solutions do not maintain labeled data that can be applied to supervised learning models .

Second, most of the learning features that are theoretically used in each study are not generalized features in the real world, because they are not contained in common network security systems. Hence, it makes difficult to utilize to practical cases. Recent efforts on intrusion detection research have considered an automation approach with deep learning technologies, and performance has been evaluated using well known datasets like NSLKDD, CICIDS2017, and Kyoto-Honey pot. However, many previous studies used benchmark dataset, which, though accurate, are not generalizable to the real world because of the insufficient features. To overcome these limitations, an employed learning model requires to evaluate with datasets that are collected in the real world.

1.2 DOMAIN INTRODUCTION

The Deep Neural Network (DNN) is a neural network with a certain level of complexity, a neural network with more than two layers. Deep neural networks use sophisticated mathematical modelling to process data in complex ways.

A neural network, in general, is a technology built to simulate the activity of the human brain – specifically, pattern recognition and the passage of input through various layers of simulated neural connections.

Many experts define deep neural networks as networks that have an input layer, an output layer and atleast one hidden layer in between .Each layer performs specific types of sorting and ordering in a process that‟s o me refer to as “featurehierarchy.” One of the keys uses of these sophisticated neural networks is dealing with unlabelled or unstructured data. The phrase “deep learning” is also used to describe these deep neural networks, as deep learning represents a specific form of machine learning where technologies using aspects of artificial intelligence seek to classify and order information in ways that go beyond simple input/output protocols

1.3 BENFITS OF NEURAL NETWORKS

Neural networks use randomness by design to ensure they effectively learn the function being approximated for the problem. Randomness is used because this class of machine learning algorithm performs better with it than without.

The most common form of randomness used in neural networks is the random initialization of the network weights. Although randomness can be used in other areas, here is just a short list:

- Randomness in Initialization, such as weights.
- Randomness in Regularization, such as dropout.
- Randomness in Layers, such as word embedding

CHAPTER 2

LITERATURE SURVEY

2.1 Enhanced Network Anomaly Detection Based on Deep Neural Networks

Due to the monumental growth of Internet applications in the last decade, the need for security of information network has increased manifolds. As a primary defense of network infrastructure, an intrusion detection system is expected to adapt to dynamically changing threat landscape. Many supervised and unsupervised techniques have been devised by researchers from the discipline of machine learning and data mining to achieve reliable detection of anomalies. Deep learning is an area of machine learning which applies neuron-like structure for learning tasks. Deep learning has profoundly changed the way we approach learning tasks by delivering monumental progress in different disciplines like speech processing, computer vision, and natural language processing to name a few. It is only relevant that this new technology must be investigated for information security applications. The aim of this paper is to investigate the suitability of deep learning approaches for anomaly-based intrusion detection system. For this research, we developed anomaly detection models based on different deep neural network structures, including convolutional neural networks, auto encoders, and recurrent neural networks. These deep models were trained on NSLKDD training data set and evaluated on both test datasets provided by NSLKDD ,namely NSLKDD Test+ and NSLKDDTest21. All experiments in this paper are performed by authors on a GPU- based test bed. Conventional machine learning-based intrusion detection models were implemented using well-known classification techniques, including extreme learning machine, nearest neighbor, decision-tree, random-forest, support vector machine, naive-bays, and quadratic discriminant analysis. Both deep and conventional machine learning models were evaluated using well-known classification metrics ,including receiver operating characteristics ,area under curve, precision-recall curve, mean average precision and accuracy of classification

Experimental results of deep IDS models showed promising results for real-world application in anomaly detection systems.

2.2 Network Intrusion Detection Based on Directed Acyclic Graph and Belief Rule Base

Intrusion detection is very important for network situation awareness. While a few methods have been proposed to detect network intrusion, they cannot directly and effectively utilize semi-quantitative information consisting of expert knowledge and quantitative data. Hence, this paper proposes a new detection model based on a directed acyclic graph (DAG) and a belief rule base (BRB). In the proposed model, called DAG-BRB, the DAG is employed to construct a multi-layered BRB model that can avoid explosion of combinations of rule number because of a large number of types of intrusion. To obtain the optimal parameters of the DAG-BRB model, an improved constraint covariance matrix adaption evolution strategy (CMA-ES) is developed that can effectively solve the constraint problem in the BRB.A case study was used to test the efficiency of the proposed DAG-BRB. The results showed that compared with other detection models, the DAG-BRB model has a higher detection rate and can be used in real networks.

2.3 HAST-IDS: Learning hierarchical spatial-temporal feature using deep neural networks

The development of ananomaly-based intrusion detection system(IDS) is a primary research direction in the field of intrusion detection. An IDS learns normal and anomalous behavior by analyzing network traffic and can detect unknown and new attacks. However, the performance of an IDS is highly dependent on feature design and designing a feature set that can accurately characterize network traffic is still an on going research issue. Anomaly-based IDSs also have the problem of a high false alarm rate (FAR), which seriously restricts their practical applications. In this paper, we propose a novel IDS called the hierarchical spatial-temporal features-based intrusion detection system (HAST-IDS), which first learns the low-level spatial features of network traffic using deep convolutional neural networks (CNNs) and then learns high-level temporal features using long short-term memory networks. The entire process of feature learning is completed by the deep neural network automatically; no feature engineering techniques are required.

The automatically learned traffic features effectively reduce the FAR. The standard DARPA1998 and ISCX2012 data sets are used to evaluate the performance of the proposed system. The experimental results show that the HAST-IDS outperforms other published approaches in terms of accuracy, detection rate, and FAR, which successfully demonstrates its effectiveness in both feature learning and reduction.

2.4 AIM

The aim of the project is Threat detection and response is the most important aspect of cyber security for IT organizations that depend on cloud infrastructure. Threat detection, therefore, describes the ability of IT organizations to quickly and accurately identify threats to the network or to applications or other assets within the network.

2.5 SCOPE OF THE PROJECT

A learning-based method geared toward determining whether an attack occurred in a large amount of data can be useful to analysts who need to instantly analyze numerous events. According to,information security solutions generally fall into two categories: analyst-driven and machine learning-drivensolutions.

2.6 OBJECTIVES

Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant Thus the objective of input design is to create an input layout that is easy to follow.

DEEP LEARNING

INTRODUCTION TO DEEP LEARNING:

- Deep structured learning or hierarchical learning or deep learning in short is part of the family of machine learning methods which are themselves a subset of the broader field of Artificial Intelligence.
- Deep learning is a class of machine learning algorithms that use several layers of nonlinear processing units for feature extraction and transformation. Each successive layer uses the output from the previous layer as input.
- Deep neural networks, deep belief networks and recurrent neural networks have been applied to fields such as computer vision, speech recognition, natural language processing, audio recognition, social network filtering, machine translation, and bioinformatics where they produced results comparable to and in some cases better than human experts have.

DEEP LEARNING ALGORITHMS AND NETWORKS:

- These are based on the unsupervised learning of multiple levels of features or representations of the data. Higher-level features are derived from lower level features to form a hierarchical representation.
- use some form of gradient descent for training.

DEEP LEARNING ARCHITECTURES:

- **Deep Neural Network**: It is a neural network with a certain level of complexity (having multiple hidden layers in between input and output layers). They are capable of modeling and processing non-linear relationships.
- **Deep Belief Network(DBN) :** It is a class of Deep Neural Network. It is multi-layer belief networks.

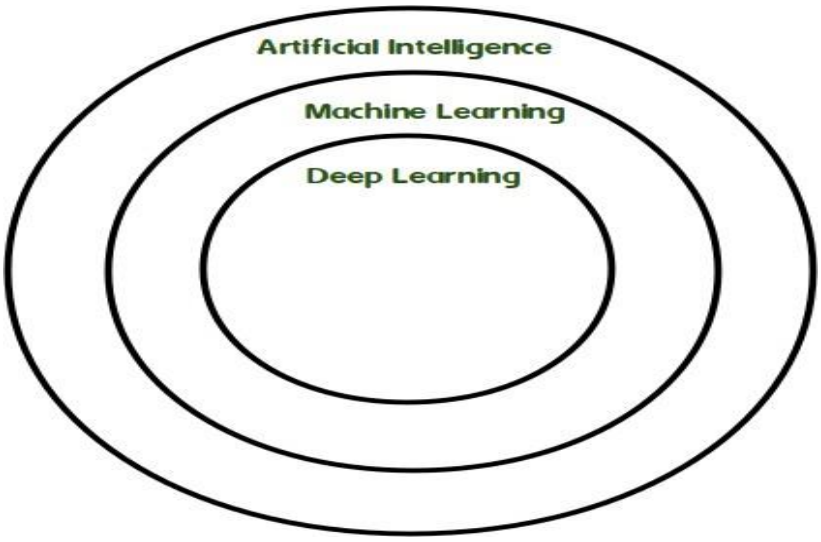
STEPS FOR PERFORMING DBN:

a. Learn a layer of features from visible units using Contrastive Divergence algorithm.

b. Treat activations of previously trained features as visible units and then learn features offeatures.

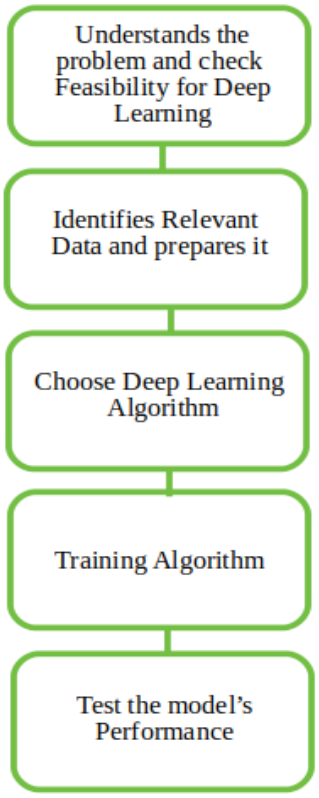
c. Finally, the whole DBN is trained when the learning for the final hidden layer is achieved.

- **Recurrent Neural Network** – Allows for parallel and sequential computation. Similar to the human brain (large feedback network of connected neurons). They are able to remember important things about the input they received and hence enables them to be more precise.



WORKING:

- First, we need to identify the actual problem in order to get the right solution and it should be understood, the feasibility of the Deep Learning should also be checked (whether it should fit Deep Learning or not).
- Second, we need to identify the relevant data which should correspond to the actual problem and should be prepared accordingly.
- Third, Choose the Deep Learning Algorithm appropriately.
- Fourth, Algorithm should be used while training the dataset. Fifth, Final testingshould be done on the dataset.



CHAPTER 3

SYSTEM ANALYSIS

- **H/W System Configuration:-**

  - Processor - Intel i3 or higher
  - RAM - 4 GB(min)
  - Hard Disk - 500 GB
  - Key Board - Standard Windows Keyboard
  - Mouse - Two or Three Button Mouse
  - Monitor - TFT/LCD

SOFTWARE REQUIREMENTS:

- **Operating system** : Windows 10 Ultimate.
- **Coding Language** : Python.

Existing System

As there is no staff accessible in automated cafés, it is hard for the eatery the board to appraise how the idea and the food is capable by the clients.
Existing Rating frameworks, like Google and Trip Advisor, just in part tackle this issue, as they just cover a piece of the client's conclusions. These rating frameworks are just utilized by a subset of the clients who rate the café on free evaluating stages on their own drive. This applies basically to clients who experience their visit as certain or negative.

In order to solve the above problem, all customers must be motivated to give a rating. This paper introduces an approach for a restaurant rating system that asks every customer for a rating after their visit to increase the number of ratings as much as possible.

This system can be used unmanned restaurants; the scoring system is based on facial expression detection using pretrained convolutional neural network (CNN)models. It allows the customer to rate the food by taking or capturing a picture of his face that reflects the corresponding feelings. Compared to text-based rating system, there is much less information and no individual experience reports collected. However, this simple fast and playful rating system should give a wider range of opinions about the experiences of the customers with the restaurant concept.

The existing cyber-threat detection systems mostly rely on conventional **machine learning techniques** such as:

- Support Vector Machines (SVM)
- K-Nearest Neighbors (k-NN)
- Random Forest (RF)
- Decision Tree (DT)
- Naïve Bayes (NB)

These systems analyze raw network traffic or event logs and attempt to classify them into normal or attack categories. Traditional methods are highly dependent on manual feature engineering, limited scalability, and often fail to adapt to evolving cyber-attack strategies. They mainly focus on signature based or rule-based detection, which cannot efficiently capture unknown and zero-day attacks.

Disadvantages of Existing System

- **SVM**: Not suitable for large datasets, and scalability is a major challenge.
- **k-NN**: Performance depends heavily on the quality of data; prediction can be slow due to high computation at query time.
- **Decision Tree**: Highly unstable, as a small change in data can significantly alter the tree structure.
- **Random Forest**: Slow in processing and unsuitable for real-time predictions.
- **Naïve Bayes**: Assumes independence among predictors, which is unrealistic for network events.
- **Overall Issue**: Conventional models are less effective in detecting advanced and evolving threats, often leading to high false positives/negatives.

Proposed System

The proposed system introduces an **AI-driven SIEM (Security Information and Event Management) framework** that uses **Artificial Neural Networks (ANNs)** with event profiling for enhanced cyber threat detection.

Key highlights:

1. **Event Profiling** – Converts raw security events into structured event profiles, enabling efficient preprocessing.
2. **Feature Extraction with TF-IDF** – Converts events into event vectors, distinguishing between normal and attack signatures.
3. **Deep Learning Models** – Utilizes **Fully Connected Neural Networks (FCNNs), Convolutional Neural Networks (CNNs), and Long Short-Term Memory (LSTM)** networks for accurate classification of threats.
4. **Performance Metrics** – Evaluates models using Precision, Recall, F-Measure, and Prediction Score. The best-performing model is deployed for real-world intrusion detection.
5. **Benchmark & Real-World Datasets** – Tested on **NSL-KDD, CICIDS2017, and real-world collected datasets**, ensuring reliability and adaptability to actual cybersecurity environments.

Advantages of Proposed System

- **Improved Accuracy** – Deep learning (CNN, LSTM) models significantly outperform conventional ML models.
- **Automated Feature Extraction** – CNNs automatically detect important features without manual intervention.
- **Better Temporal Analysis** – LSTMs handle sequential data efficiently, making them well-suited for analyzing attack patterns over time.
- **Reduced False Positives** – Helps security analysts quickly filter out irrelevant alerts and focus on real threats.
- **Scalability** – Can handle large-scale datasets and real-time traffic.
- **Deployment-Ready** – The trained model can be deployed in real-world SIEM systems for automated cyber threat detection.

CHAPTER 4
FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are,

- **ECONOMICALFEASIBILITY**
- **TECHNICALFEASIBILITY**
- **SOCIALFEASIBILITY**

4.1 ECONOMIC FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount off and that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

4.2 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

4.3 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system**.**

4.4 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those Steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy.

4.5 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system‟s relationship to help user decision- making.

Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

1. Select methods for presenting information.
2. Create document, report, or other formats that contain information produced by the system.

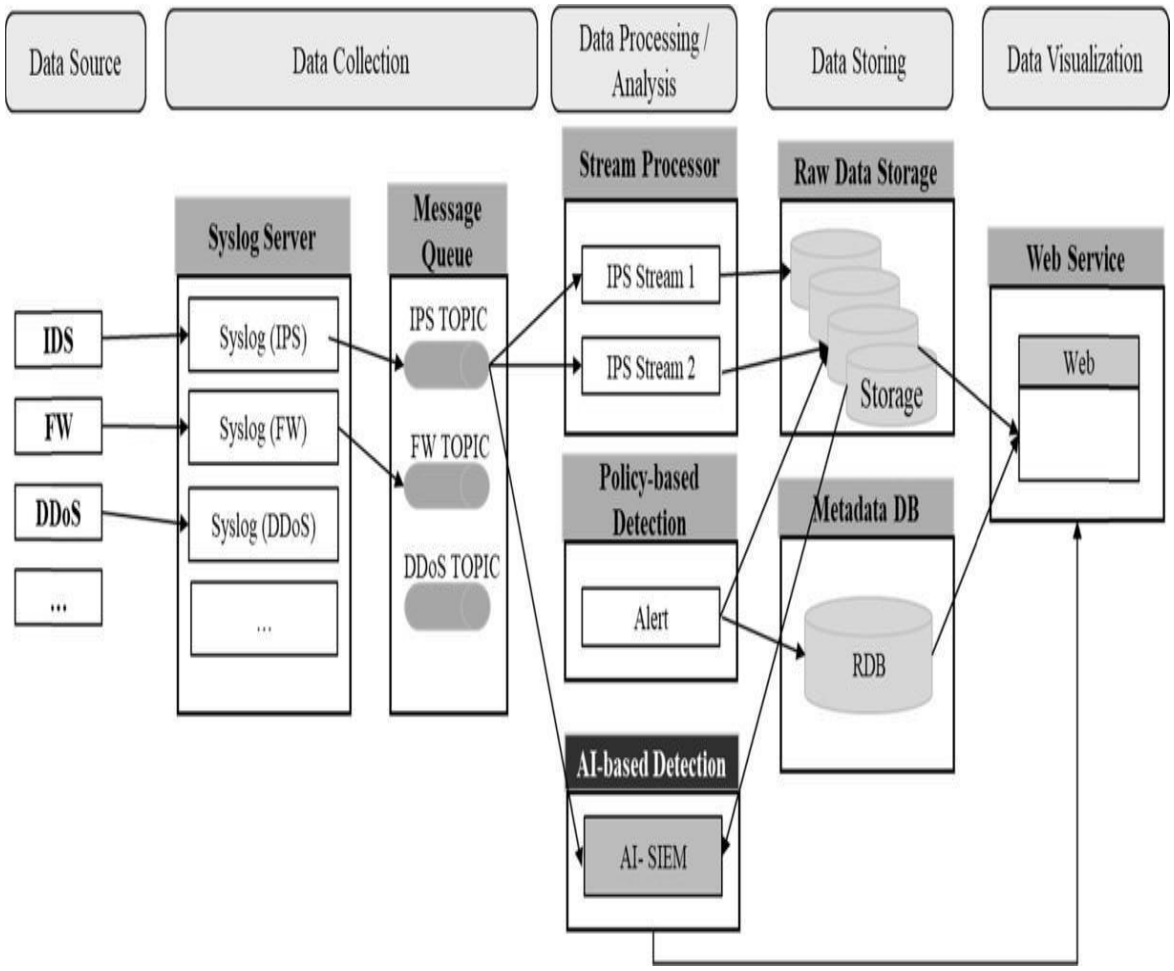
The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

4.5 MODULES

1. **Data Parsing**: This module takes input dataset and parse that dataset to create a raw data event model
2. **TF-IDF**: using this module we will convert raw data into event vector which will contains normal and attack signatures
3. **Event Profiling Stage**: Processed data will be spitted into train and test model based on profiling events.
4. **Deep Learning Neural Network Model**: This module runs CNN and LSTM algorithms on train and test data and then generate a training model. Generated trained model will be applied on test data to calculate prediction score, Recall, Precision and FMEA sure. Algorithm will learn perfectly will yield better accuracy result and that model will be selected to deploy on real system for attack detection.

4.6 SYSTEM ARCHITECTURE



CHAPTER 5

SYSTEM DESIGN

5.1 UML INTRODUCTION

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.

UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.

OMG is continuously making efforts to create a truly industry standard.

- UML stands for **Unified Modeling Language**.
- UML is different from the other common programming languages such as C++, Java, COBOL, etc.
- UML is a pictorial language used to make software blueprints.
- UML can be described as a general-purpose visual modeling language to visualize, specify, construct, and document software system.
- Although UML is generally used to model software systems, it is not limited within this boundary. It is also used to model non-software systems as well. For example, the process flow in a manufacturing unit, etc.

UML is not a programming language but tools can be used to generate code in various languages using UML diagrams. UML has a direct relation with object oriented analysis and design. After some standardization, UML has become an OMG standard.

Goals of UML

A picture is worth a thousand words, this idiom absolutely fits describing UML. Object-oriented concepts were introduced much earlier than UML. At that point of time, there were no standard methodologies to organize and consolidate the object-oriented development. It was then that UML came into picture.

There are a number of goals for developing UML but the most important is to define some general purpose modeling language, which all modelers can use and it also needs to be made

simple to understand and use.

UML diagrams are not only made for developers but also for business users, common people, and

anybody interested to understand the system. The system can be a software or non-software system. Thus it must be clear that UML is not a development method rather it accompanies with processes to make it a successful system.

In conclusion, the goal of UML can be defined as a simple modeling mechanism to model all possible practical systems in today‟s complex environment.

Object-Oriented Concepts

UML can be described as the successor of object-oriented (OO) analysis and design.

An object contains both data and methods that control the data. The data represents the state of the object. A class describes an object and they also form a hierarchy to model the real-world system. The hierarchy is represented as inheritance and the classes can also be associated in different ways as per the requirement.

Objects are the real-world entities that exist around us and the basic concepts such as abstraction, encapsulation, inheritance, and polymorphism all can be represented using UML.

UML is powerful enough to represent all the concepts that exist in object-oriented analysis and design. UML diagrams are representation of object-oriented concepts only. Thus, before learning UML, it becomes important to understand OO concept in detail.

Following are some fundamental concepts of the object-oriented world −

- **Objects** − Objects represent an entity and the basic building block.
- **Class** − Class is the blue print of an object.
- **Abstraction** − Abstraction represents the behavior of an real world entity.
- **Encapsulation** − Encapsulation is the mechanism of binding the data together and hiding them from the outside world.
- **Inheritance** − Inheritance is the mechanism of making new classes from existing ones.
- **Polymorphism** − It defines the mechanism to exists in different forms.

OO Analysis and Design

OO can be defined as an investigation and to be more specific, it is the investigation of objects. Design means collaboration of identified objects.

Thus, it is important to understand the OO analysis and design concepts. The most important purpose of OO analysis is to identify objects of a system to be designed. This analysis is also done for an existing system. Now an efficient analysis is only possible when we are able to start thinking in a way where objects can be identified. After identifying the objects, their relationships are identified and finally the design is produced.

The purpose of OO analysis and design can described as −

- Identifying the objects of a system.
- Identifying their relationships.
- Making a design, which can be converted to executables using OO languages.

There are three basic steps where the OO concepts are applied and implemented. The steps can be defined as

OO Analysis→ OO Design→ OO implementation using OO languages

The above three points can be described in detail as −

- During OO analysis, the most important purpose is to identify objects and describe them in a proper way. If these objects are identified efficiently, then the next job of design is easy. The objects should be identified with responsibilities. Responsibilities are the functions performed by the object. Each and every object has some type of responsibilities to be performed. When these responsibilities are collaborated, the purpose of the system is fulfilled.
- The second phase is OO design. During this phase, emphasis is placed on the requirements and their fulfilment. In this stage, the objects are collaborated according to their intended association. After the association is complete, the design is also complete.
- The third phase is OO implementation. In this phase, the design is implemented using OO languages such as Java, C++, etc.

5.2 UML Diagrams

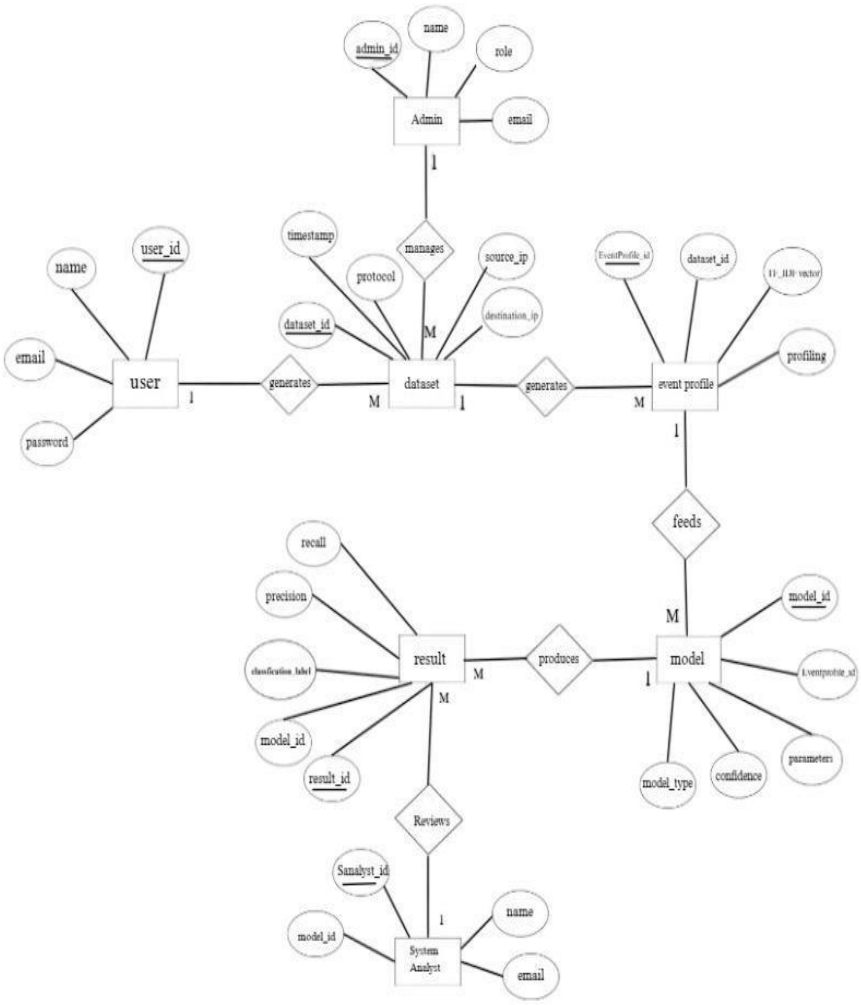
UML diagrams are the ultimate output of the entire discussion. All the elements, relationships are used to make a complete UML diagram and the diagram represents a system.

The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it complete.

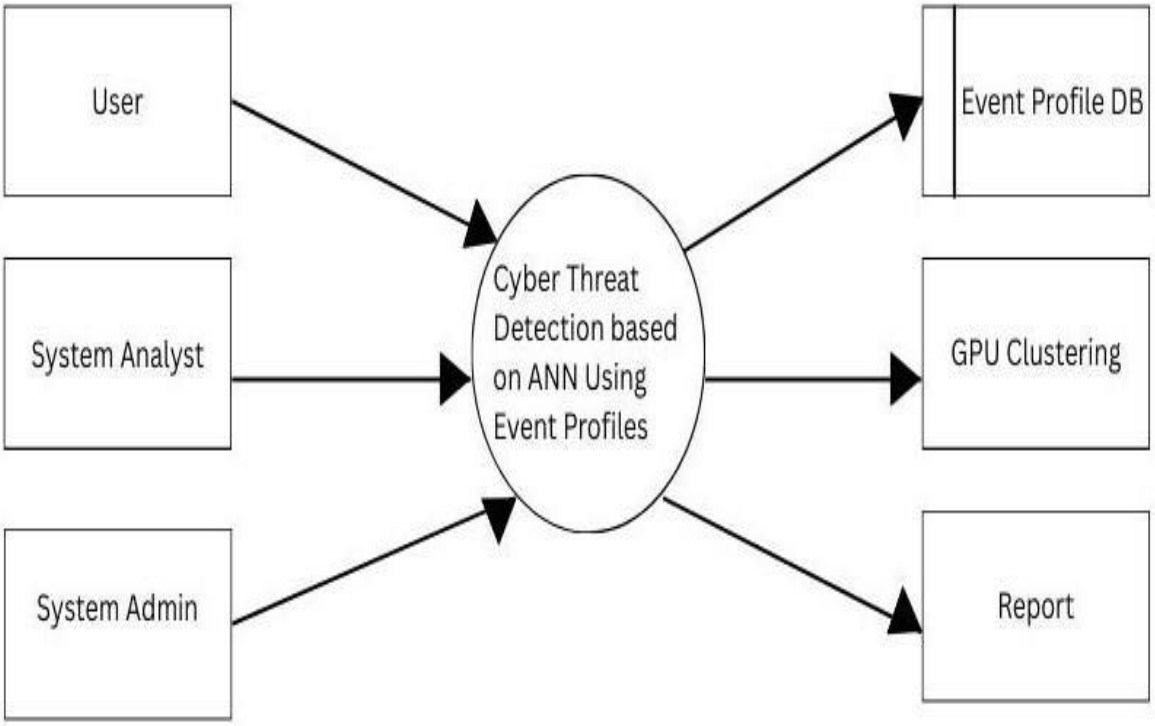
UML includes the following nine diagrams, the details of which are described in the subsequent chapters.

- Use Case diagram
- Class diagram
- Sequence diagram
- Activity diagram
- Componenet diagram
- Deployment diagram

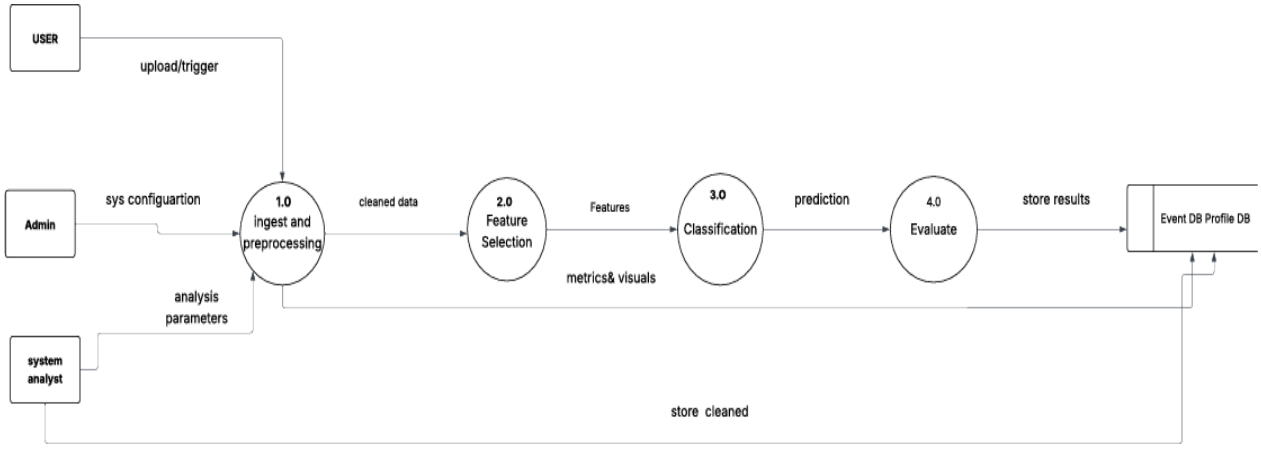
ER Diagram


admin_id
name
role
Admin
email
1
manages
M
timestamp
protocol
source_ip
dataset_id
destination_ip
dataset
user_id
name
email
password
user
1
generates
M
1
generates
EventProfile_id
dataset_id
profiling
event profile
M
1
feeds
M
model_id
model
1
parameters
confidence
model_type
recall
precision
result
M
produces
M
model_id
result_id
Reviews
1
Sanalyst_id
model_id
System Analyst
name
email

DFD Level 0


User
System Analyst
System Admin
Cyber Threat Detection based on ANN Using Event Profiles
Event Profile DB
GPU Clustering
Report

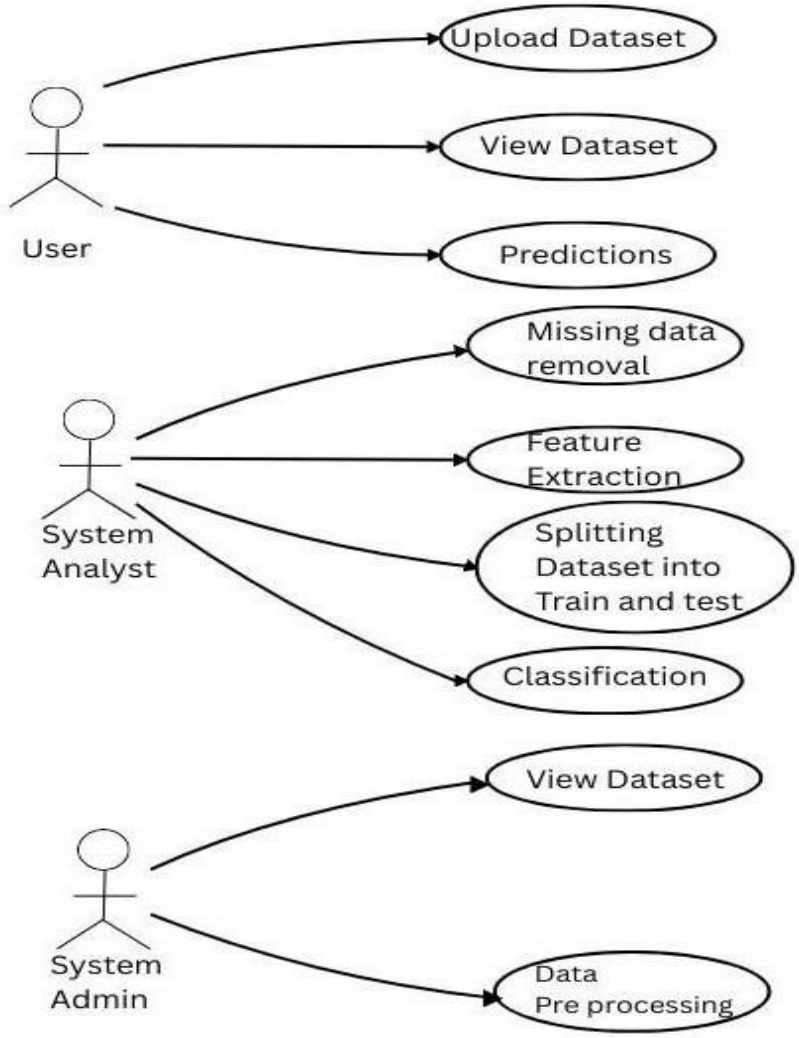
DFD Level 1


USER
upload/trigger
Admin
sys configuartion
1.0
ingest and
preprocessing
cleaned data
2.0
Feature
Selection
Features
3.0
Classification
prediction
4.0
Evaluate
store results
Event DB Profile DB
metrics& visuals
analysis
parameters
system
analyst
store cleaned

5.2.1 Use case diagram:

Use case diagrams are a set of use cases, actors, and their relationships. They represent the use case view of a system.

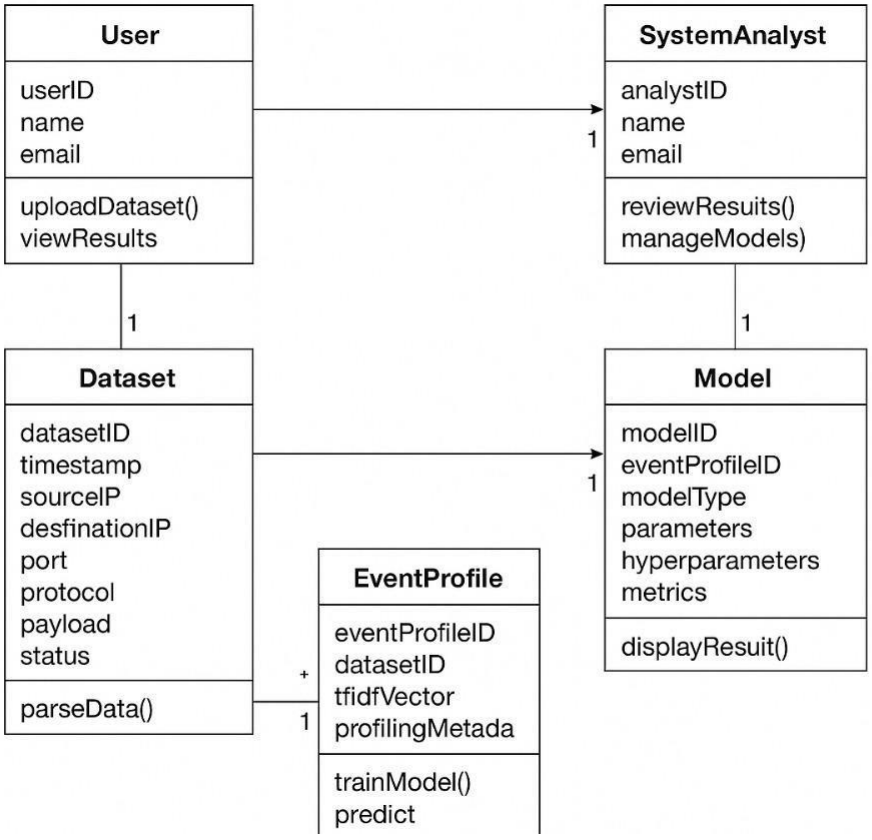
A use case represents a particular functionality of a system. Hence, use case diagram is used to describe the relationships among the functionalities and their internal/external controllers. These controllers are known as **actors**.



5.2.2 Class diagram:

Use case diagrams are a set of use cases, actors, and their relationships. They represent the use case view of a system.

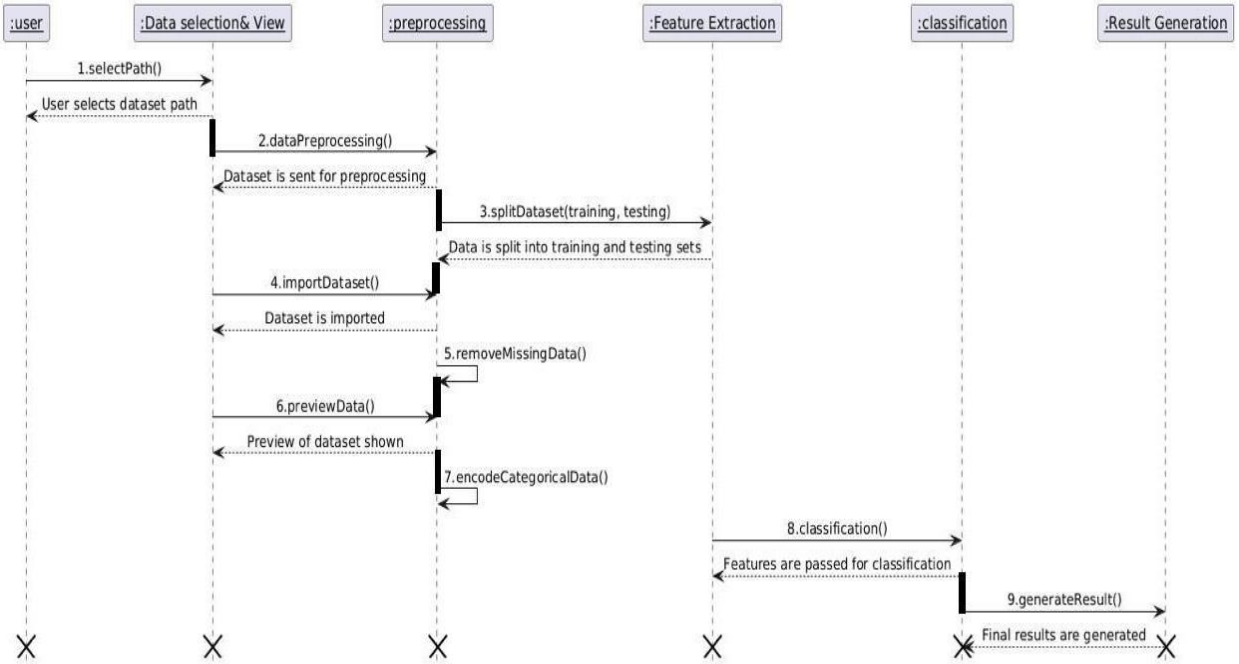
A use case represents a particular functionality of a system. Hence, use case diagram is used to describe the relationships among the functionalities and their internal/external controllers. These controllers are known as **actors**.



5.2.3 Sequence diagram:

A sequence diagram is an interaction diagram. From the name, it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.

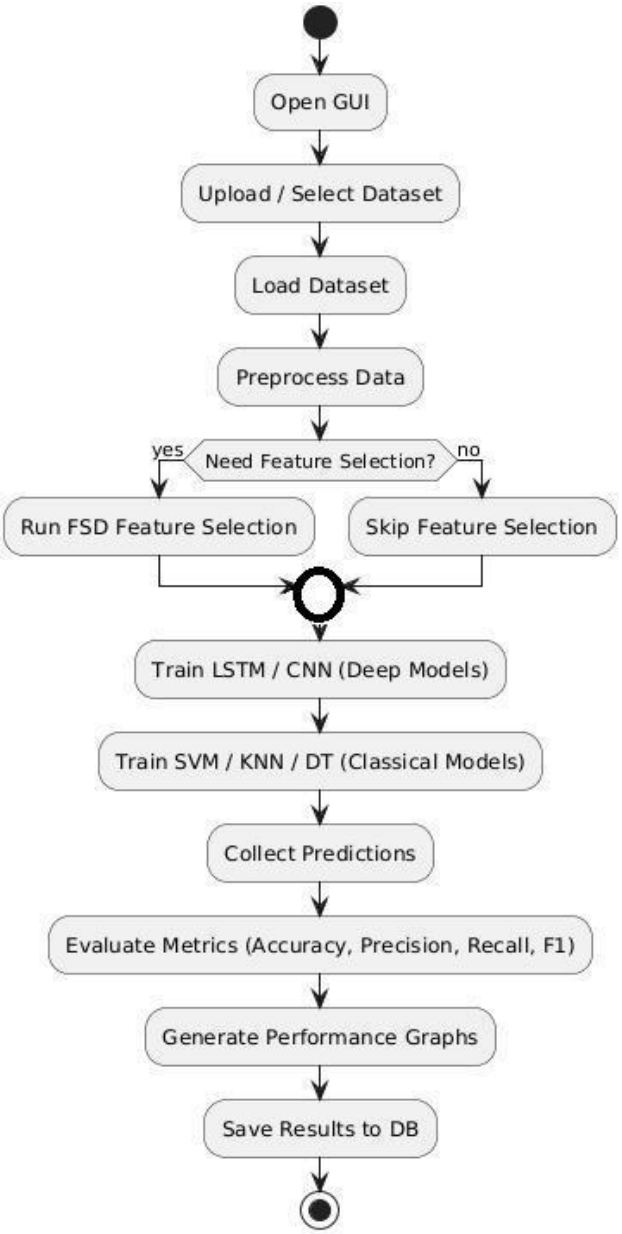
Interaction among the components of a system is very important from implementation and execution perspective. Sequence diagram is used to visualize the sequence of calls in a system to perform a specific functionality.



5.2.4 Activity diagram:

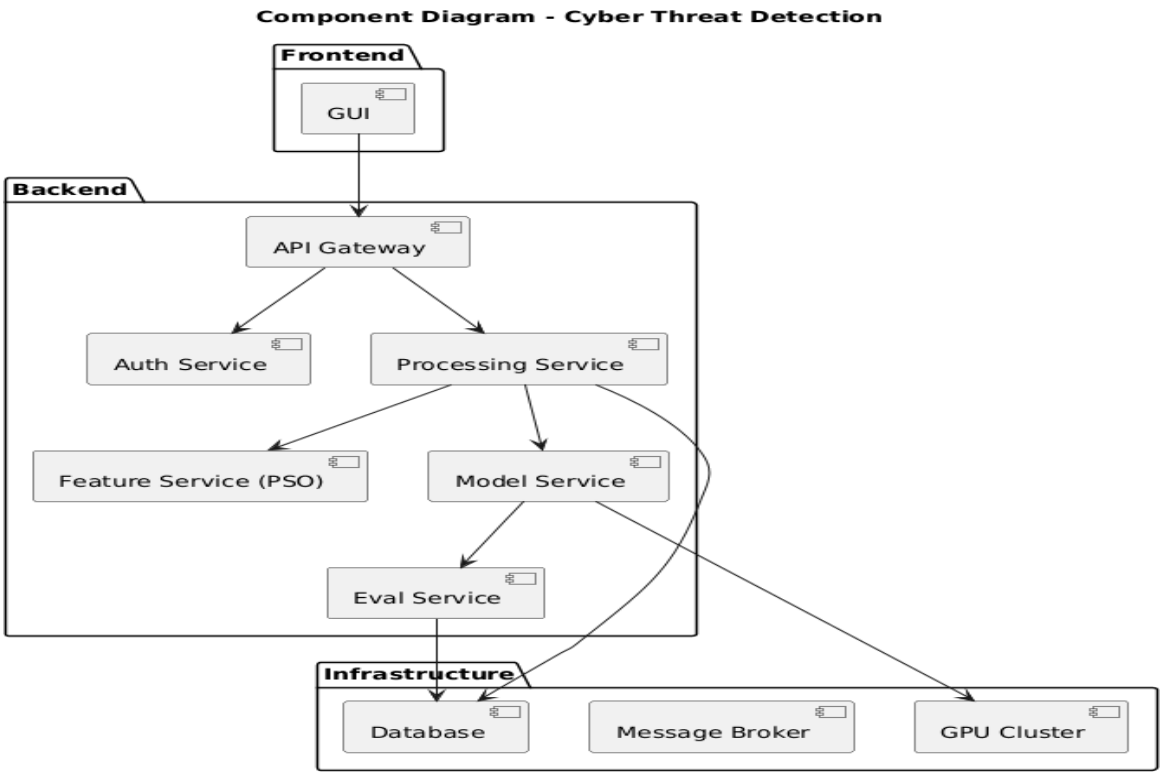
A sequence diagram is an interaction diagram. From the name, it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.

Interaction among the components of a system is very important from implementation and execution perspective. Sequence diagram is used to visualize the sequence of calls in a system to perform a specific functionality.



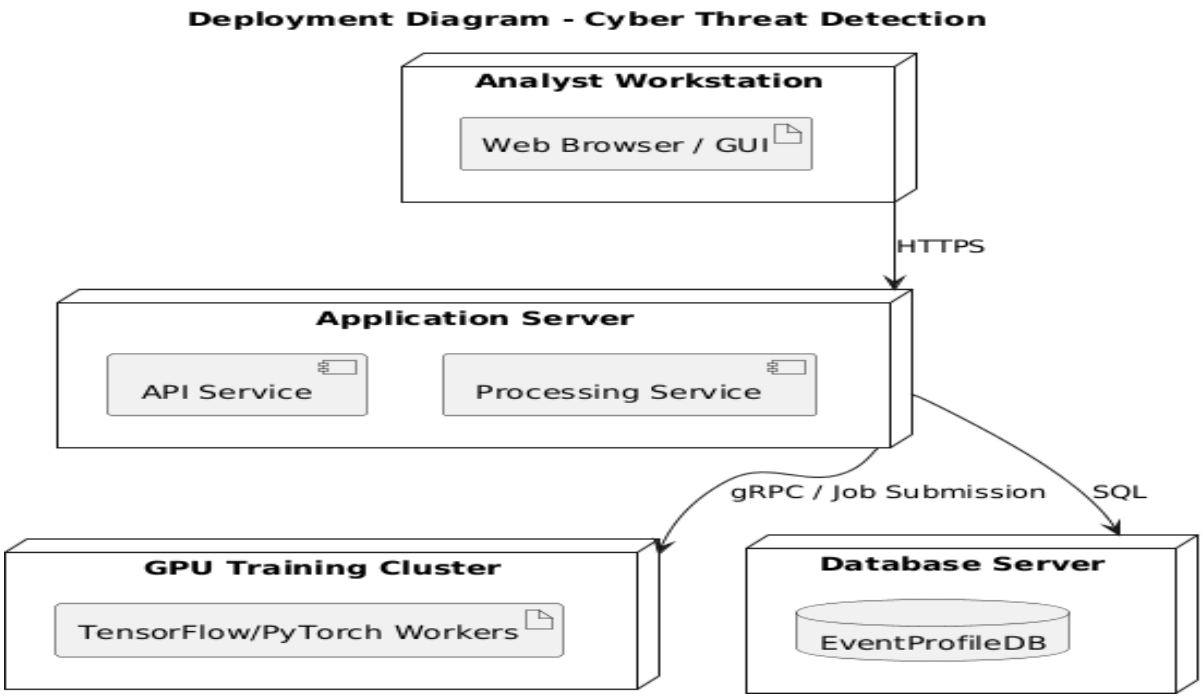
5.2.4 Component diagram:

- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations



5.2.5 Deployment diagram

- A deployment diagram shows the configuration of run-time processing nodes and the components that live on them
- Deployment diagrams address the static deployment view of an architecture.



CHAPTER 6
IMPLEMENTATION

6.1 INTRODUCTION TO PYTHON

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It provides constructs that enable clear programming on both small and large scales. In July 2018, Van Rossum stepped down as the leader in the language community.

FEATURES:

- Python uses dynamic typing, and a combination of reference counting and a cycle-detecting garbage collector for memory management. It also features dynamic name resolution (late binding), which binds method and variable names during program execution.
- Python's design offers some support for functional programming in the Lisp tradition. It has filter(), map(),and reduce() functions; list comprehensions, dictionaries, and sets; and generator expressions.
- The standard library has two modules (itertools and functools) that implement functional tools borrowed from Haskell and Standard ML. The language's core philosophy is summarized in the document The Zen of Python (PEP 20), which includes aphorisms such as:

  - Beautiful is better than ugly
  - Explicit is better than implicit
  - Simple is better than complex
  - Complex is better than complicated
  - Readability counts

LIBRARIES:

- Python's large standard library, commonly cited as one of its greatest strengths provides tools suited to many tasks. For Internet-facing applications, many standard formats and protocols such as MIME and HTTP are supported. It includes modules for creating graphical user interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary precision decimals, manipulating regular expressions, and unit testing.
- Some parts of the standard library are covered by specifications (for example, the Web Server Gateway Interface (WSGI) implementation wsgiref follows PEP 333, but most modules are not. They are specified by their code, internal documentation, and test suites (if supplied). However, because most of the standard library is cross-platform Python code, only a few modules need altering or rewriting for variant implementations.
- As of March 2018, the Python Package Index (PyPI), the official repository for third-party Python software, contains over 130,000 packages with a wide range of functionality, including:

  - Graphical user interfaces
  - Web frameworks
  - Multimedia
  - Databases
  - Networking
  - Automation
  - Web scraping
  - Documentation
  - System administration
  - Scientific computing
  - Text processing
  - Image processing

6.2 TECHNOLOGIES USED

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++ or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. Python, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management.

It supports multiple programming paradigms, including object oriented, imperative, functional and has a large and comprehensive standard library.

Python Installations:

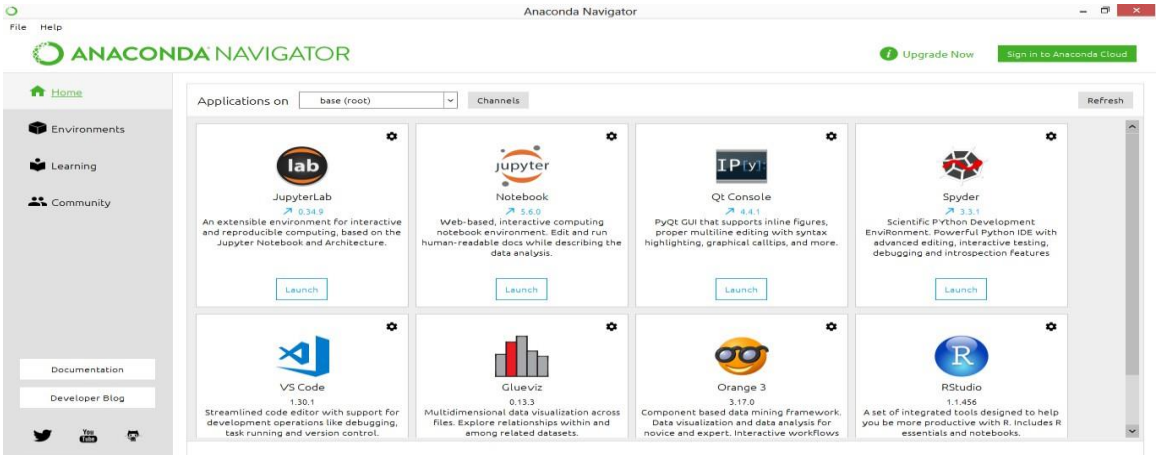
- Python Installation
- IDLE Introduction
- Pycharm Installation
- PIP Installation
- Jupyter Notebook Installation

URL:

Go to www.python.org/downloads

1. Anaconda

- **Anaconda** is a free and open-source distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify package management and deployment. Package versions are managed by the package management system conda. The Anaconda distribution is used by over 6 million users and includes more than 1400 popular data-science packages suitable for Windows, Linux, and MacOS.
- Anaconda distribution comes with more than 1,400 packages as well as the Conda package and virtual environment manager, called Anaconda Navigator, so it eliminates the need to learn to install each library independently.
- The open source packages can be individually installed from the Anaconda repository with the `conda install` command or using the `pip install` command that is installed with Anaconda. Pip packages provide many of the features of conda packages and in most cases they can work together .Custom packages can be made using the `conda build` command, and can be shared with others by uploading them to Anaconda Cloud, PyPI or other repositories.
- The default installation of Anaconda2 includes Python 2.7 and Anaconda3 includes Python 3.7. However, you can create new environments that include any version of Python packaged with conda.



Anaconda Navigator

Anaconda Navigator

- Anaconda Navigator is a desktop graphical user interface (GUI) included in Anaconda distribution that allows users to launch applications and manage conda packages, environments and channels without using command-line commands. Navigator can search for packages on Anaconda Cloud or in a local Anaconda Repository, install them in an environment, run the packages and update them. It is available for Windows, macOS and Linux.
- The following applications are available by default in Navigator:

  - JupyterLab
  - Jupyter Notebook
  - QtConsole
  - Spyder
  - Glueviz
  - Orange
  - Rstudio
  - Visual Studio Code

Conda:

- Conda is an open source, cross-platform, language-agnostic package manager and environment management system that installs, runs, and updates packages and their dependencies. It was created for Python programs, but it can package and distribute software for any language (e.g., R), including multi-language projects. The Conda package and environment manager is included in all versions of Anaconda, Mini conda, and Anaconda Repository.

Anaconda Cloud:

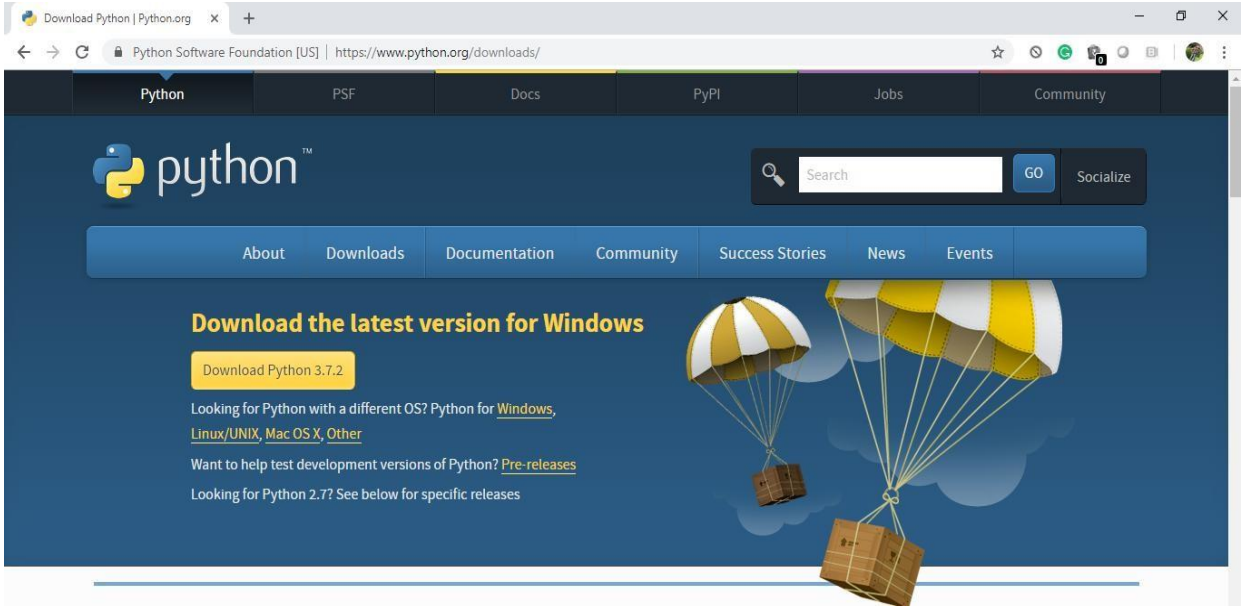
- Anaconda Cloud is a package management service by Anaconda where you can find, access, store and share public and private notebooks, environments, and conda and PyPI packages. Cloud hosts useful Python packages, notebooks and environments for a wide variety of applications. You do not need to log in or to have a Cloud account, to search for public packages, download and install them.

2. Jupyter Notebook

- The notebook extends the console-based approach to interactive computing in a qualitatively new direction, providing a web-based application suitable for capturing the whole computation process: developing, documenting, and executing code, as well as communicating the results.
- The Jupyter notebook combines two components:

A web application: A browser-based tool for interactive authoring of documents which combine explanatory text, mathematics, computations and their rich media output.

Notebook documents: A representation of all content visible in the web application, including inputs and outputs of the computations, explanatory text, mathematics, images, and rich media representations of objects.

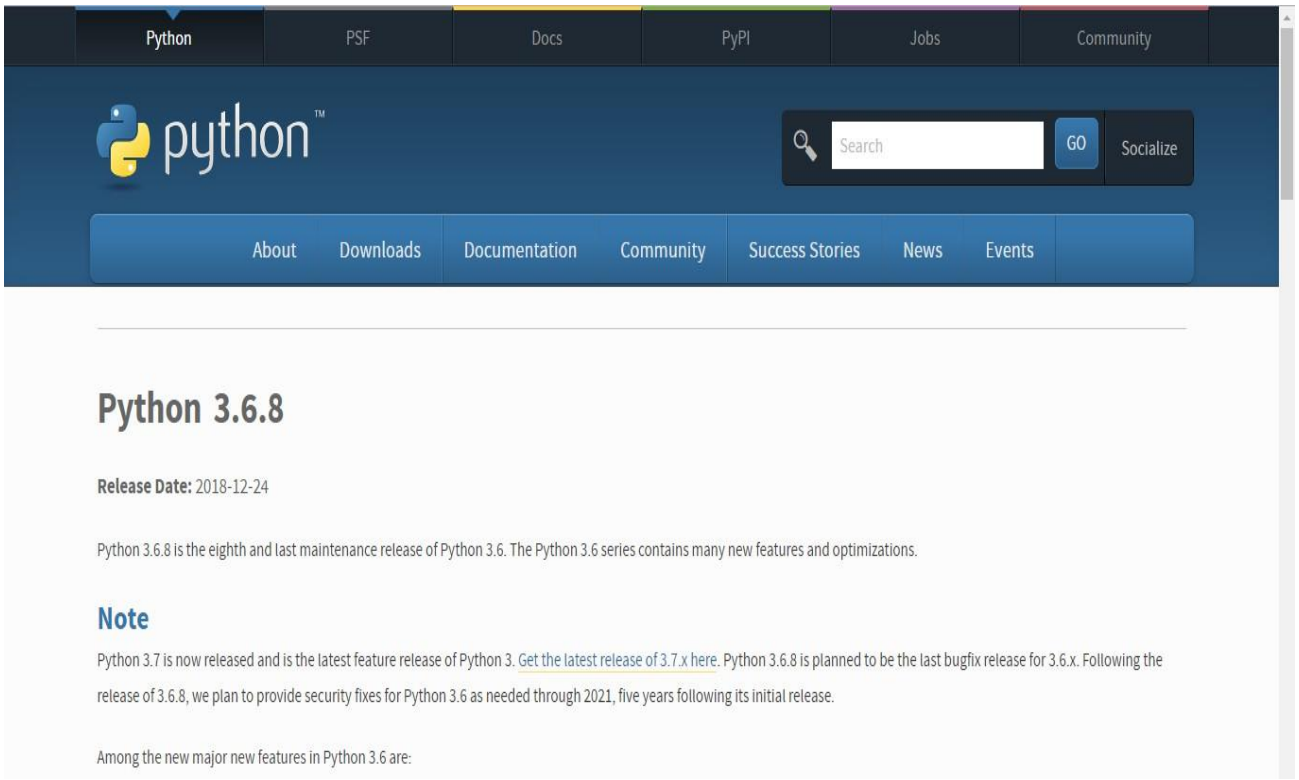

Download Python | Python.org
Python Software Foundation [US] | https://www.python.org/downloads/
Python
PSF
Docs
PyPI
Jobs
Community
python
Search
GO
Socialize
About
Downloads
Documentation
Community
Success Stories
News
Events
Download the latest version for Windows
Download Python 3.7.2
Looking for Python with a different OS? Python for Windows, Linux/UNIX, Mac OS X, Other
Want to help test development versions of Python? Pre-releases
Looking for Python 2.7? See below for specific releases

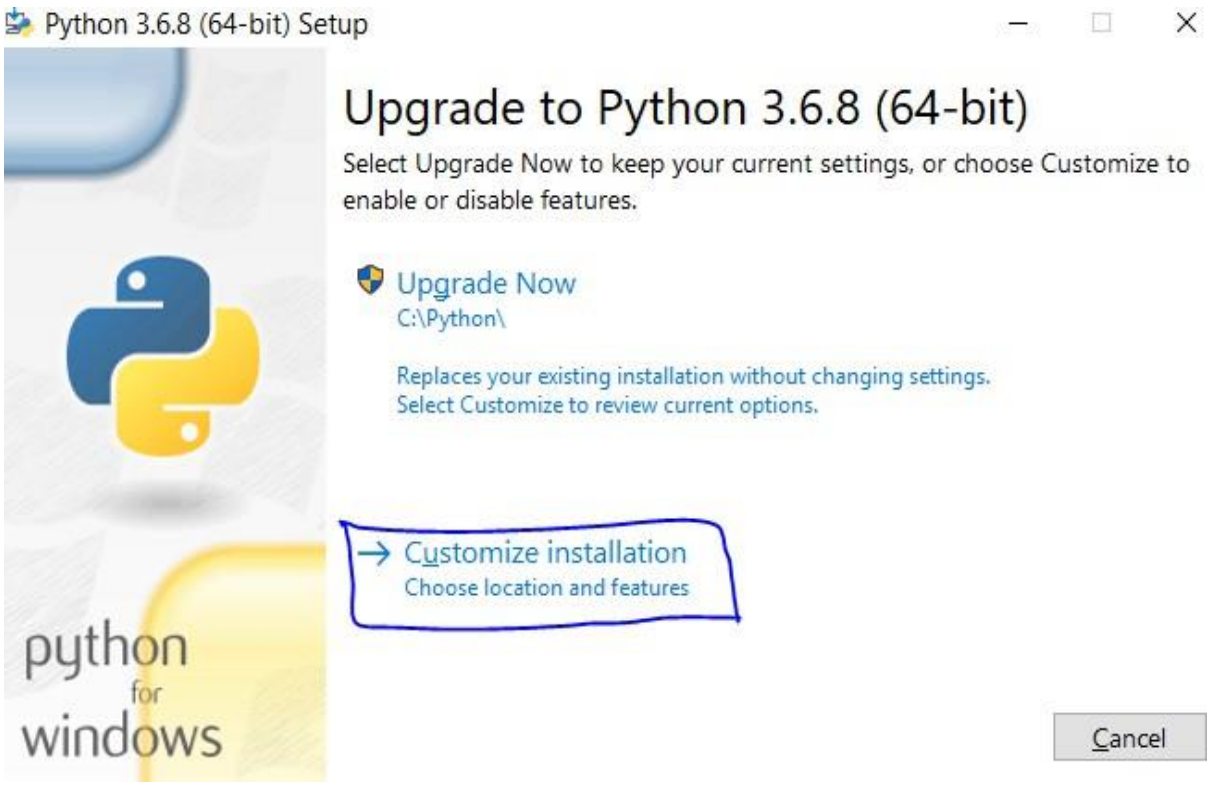
Looking for a specific release?

Python releases by version number:

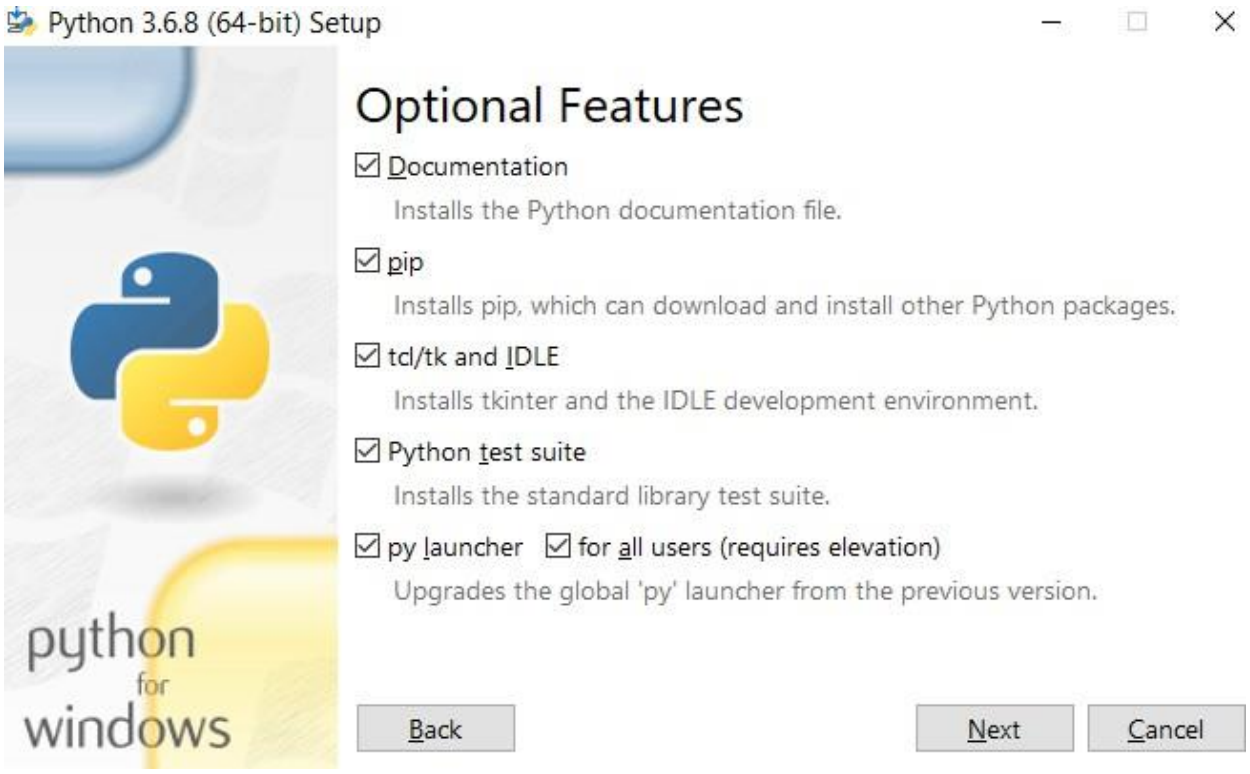
Release version	Release date		Click for more
Python 3.7.2	2018-12-24	Download	Release Notes
Python 3.6.8	2018-12-24	Download	Release Notes
Python 3.7.1	2018-10-20	Download	Release Notes
Python 3.6.7	2018-10-20	Download	Release Notes
Python 3.5.6	2018-08-02	Download	Release Notes
Python 3.4.9	2018-08-02	Download	Release Notes
Python 3.7.0	2018-06-27	Download	Release Notes
Python 3.6.6	2018-06-27	Download	Release Notes

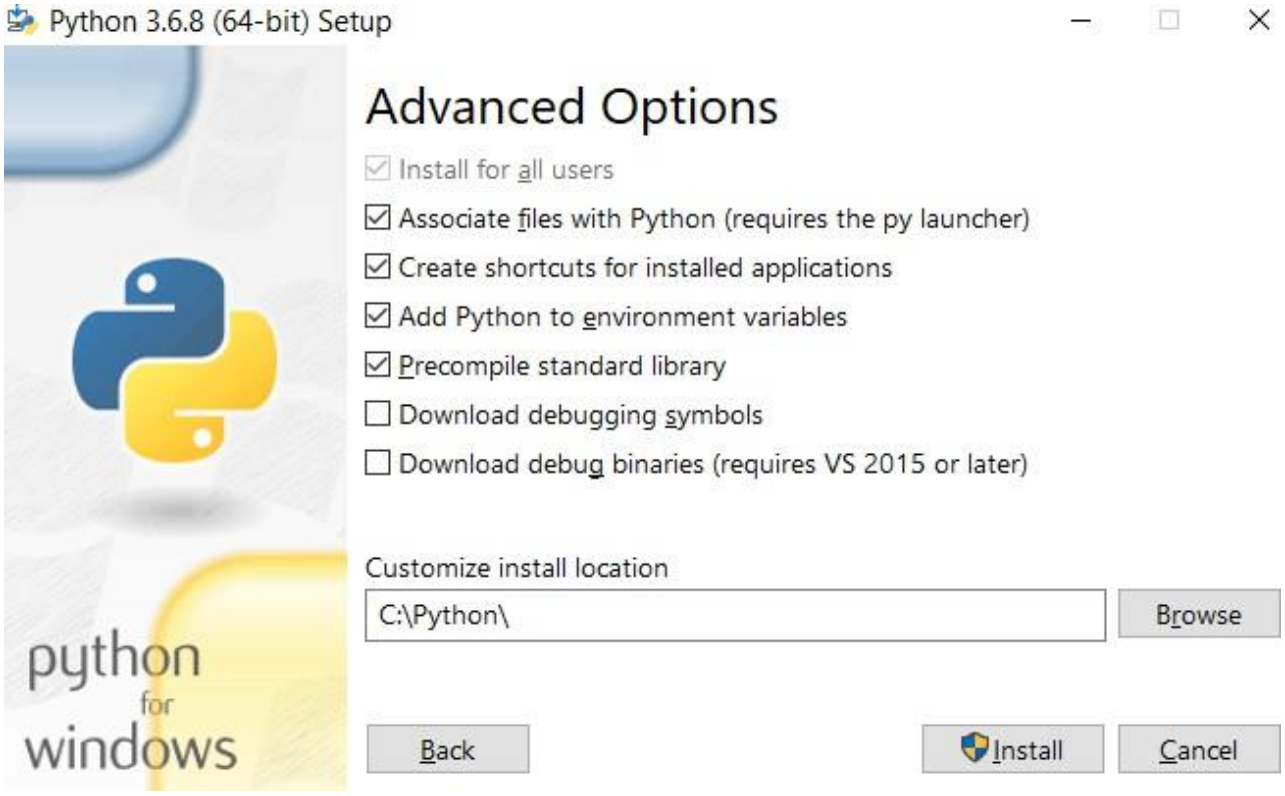
View older releases


Python
PSF
Docs
PyPI
Jobs
Community
python™
Search
GO
Socialize
About
Downloads
Documentation
Community
Success Stories
News
Events
Python 3.6.8
Release Date: 2018-12-24
Python 3.6.8 is the eighth and last maintenance release of Python 3.6. The Python 3.6 series contains many new features and optimizations.
Note
Python 3.7 is now released and is the latest feature release of Python 3. Get the latest release of 3.7.x here. Python 3.6.8 is planned to be the last bugfix release for 3.6.x. Following the release of 3.6.8, we plan to provide security fixes for Python 3.6 as needed through 2021, five years following its initial release.
Among the new major new features in Python 3.6 are:



- Click on the .exe file
- I get upgrade as I already have python
- You will get Install now.
- Click on customize installation.


Python 3.6.8 (64-bit) Setup
Optional Features
Documentation
Installs the Python documentation file.
pip
Installs pip, which can download and install other Python packages.
tcl/tk and IDLE
Installs tkinter and the IDLE development environment.
Python test suite
Installs the standard library test suite.
py launcher
for all users (requires elevation)
Upgrades the global 'py' launcher from the previous version.
python
for
windows
Back
Next
Cancel



- Change the default path by selecting browse.
- Select a well-known path which you can access.
- Do not select drive just because I selected.
- Make sure all are ticked as in image.

Type in „python“ in the cmd.

- You will get the python version and you are in python shell now.
- The three greater than symbols indicate you are in python shell
- To come out of python shell enter ctrl+z and enter or type in „exit()“.

```
Command Prompt - python
Microsoft Windows [Version 10.0.17134.112]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Suryavamsi>python
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>


C:\Users\Suryavamsi>python
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()

C:\Users\Suryavamsi>
```

Enter „pip --version" in the command line

- You will get the pip installed path and the version.
- Don"t worry if versions do not match. We can upgrade pip later.

```
C:\Users\Suryavamsi>pip --version
pip 10.0.1 from c:\python\lib\site-packages\pip (python 3.6)
```

Installing Jupyter Notebook

- Upgrading pip to the latest . Type in the command „python3 –m pip install -- upgrade pip
- Type the command „pip install Jupyter" in the command prompt.

```
C:\Users\Suryavamsi\Desktop>pip install jupyter
Collecting jupyter
```

- Jupyter should successfully install. Donk worry abt versions.

```
Installing collected packages: jupyter
Successfully installed jupyter-1.0.0

C:\Users\Suryavamsi\Desktop>
```

- Type in jupyter and a space and notebook. Jupyter will open.

```
C:\Users\Suryavamsi\Desktop>jupyter notebook
```

- Jupyter note book should open in the browser.

Jupyter Notebook:

- By default jupyter notebook will be showing you your C drive user data or from the path where we opened the book.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open source, BSD-licensed library providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language. We can analyze data in pandas with:

- Series
- Data Frames

Series is one dimensional (1-D) array defined in pandas that can be used to store any data type.

Data Frames is two-dimensional (2-D) data structure defined in pandas which consists of rows and columns.

Matplotlib:

Matplot lib is a python library used to create 2D graphs and plots by using python scripts. It has a module named pyplot which makes things easy for plotting by providing feature to control line styles, font properties, formatting axes etc. It supports a very wide variety of graphs and plots namely histogram, bar charts, power spectra, error charts etc. It is used along with NumPy to provide an environment that is an effective open source alternative for MatLab. It can also be used with graphics toolkits like PyQt and wxPython.

scikit-learn:

Scikit-learn is a machine learning library for Python. It features several regression, classification and clustering algorithms including SVMs, gradient boosting, k-means, random forests and DBSCAN. Scikit is written in Python (most of it) and some of its core algorithms are written in Cython for even better performance.

Scikit-learn is used to build models and it is not recommended to use it for reading, manipulating and summarizing data as there are better frameworks available for the purpose.

It is open source and released under BSD license.

Seaborn:

Seaborn is an open source, BSD-licensed Python library providing high level API for visualizing the data using Python programming language. Data can be visualized by representing it as plots which is easy to understand, explore and grasp. Such data helps in drawing the attention of key elements.

To analyse a set of data using Python, we make use of Matplotlib, a widely implemented 2D plotting library. Likewise, Seaborn is a visualization library in Python. It is built on top of Matplotlib.

Python is a **high-level, interpreted**, **interactive** and **object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

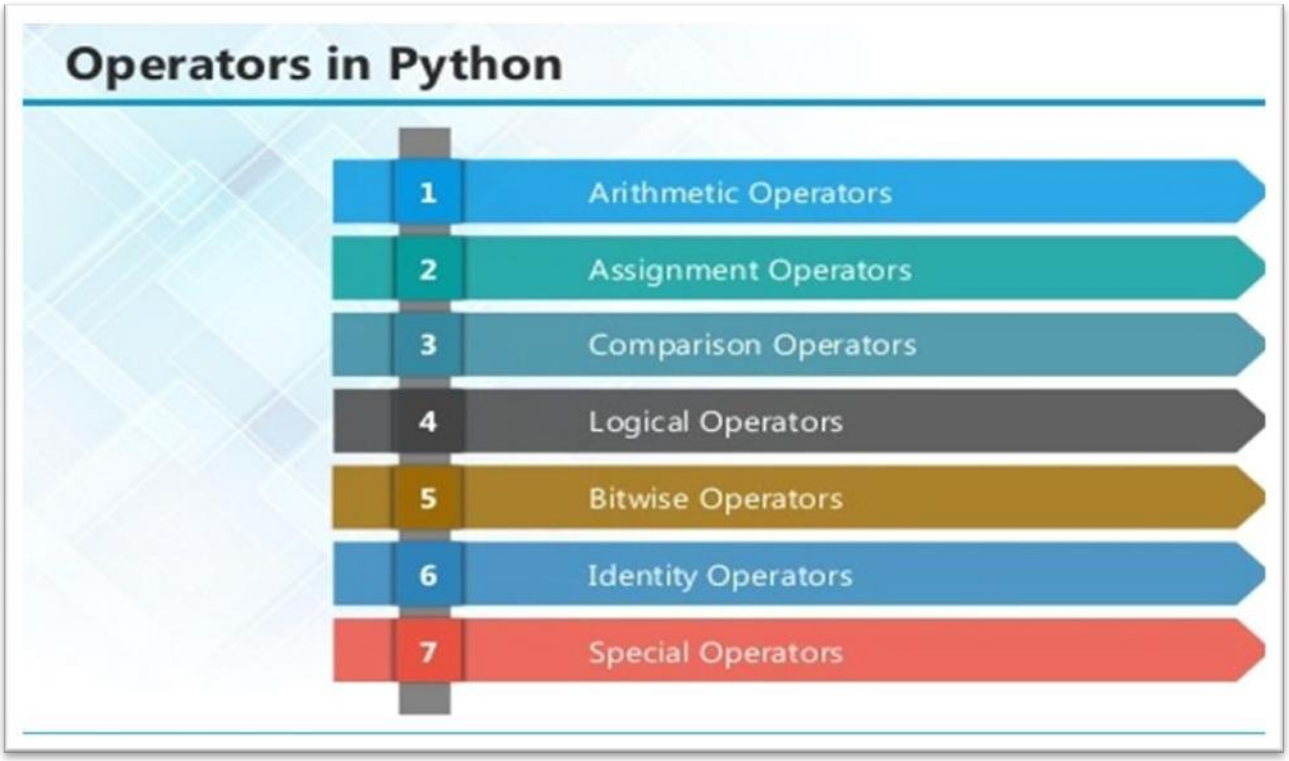
Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



6.3.1 ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	a + b = 30
- Subtraction	Subtracts right hand operand from left hand operand.	a – b = -10
* Multiplication	Multiplies values on either side of the operator	a * b = 200
/ Division	Divides left hand operand by right hand operand	b / a = 2
% Modulus	Divides left hand operand by right hand operand and returns remainder	b % a = 0
** Exponent	Performs exponential (power) calculation on operators	a**b =10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	9//2 = 4 and 9.0//2.0 = 4.0, -11//3 = -4, -11.0//3 = -4.0

6.3.2 ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	c = a + b assigns value of a + b into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	c += a is equivalent to c = c + a
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	c -= a is equivalent to c = c - a
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	c *= a is equivalent to c = c * a
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	c /= a is equivalent to c = c / ac /= a is equivalent to c = c / a

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	c %= a is equivalent to c = c % a
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	c **= a is equivalent to c = c ** a
//= Floor Division	It performs floor division on operators and assign value to the left operand	c //= a is equivalent to c = c // a

6.3.3 IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here **is** results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here **is not** results in 1 if id(x) is not equal to id(y

6.3.4 COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
| Binary OR	It copies a bit if it exists in either operand.	(a | b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

6.3.5 LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

6.3.6 Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>><<	Right and left bitwise shift
&	Bitwise 'AND'
^ |	Bitwise exclusive `OR' and regular `OR'
<= <>>=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 =['physics','chemistry',1997,2000];
list2 =[1,2,3,4,5];
list3 =["a","b","c","d"]
```

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	**Results**	**Description**
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions −

SN	Function with Description
1	cmp(list1, list2) Compares elements of both lists.
2	len(list) Gives the total length of the list.
3	max(list) Returns item from the list with max value.
4	min(list) Returns item from the list with min value.
5	list(seq) Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	list.append(obj) Appends object obj to list
2	list.count(obj) Returns count of how many times obj occurs in list
3	list. extend(seq) Appends the contents of seq to list
4	list.index(obj) Returns the lowest index in list that obj appears
5	list.insert(index, obj) Inserts object obj into list at offset index
6	list.pop(obj=list[-1]) Removes and returns last object or obj from list
7	list.remove(obj) Removes object obj from list
8	list.reverse() Reverses objects of list in place
9	list.sort([func]) Sorts objects of list, use compare function if given

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example −

```
tup1 =('physics','chemistry',1997,2000);

tup2 =(1,2,3,4,5);

tup3 ="a","b","c","d";
```

The empty tuple is written as two parentheses containing nothing −

```
tup1 =();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value −

```
tup1 =(50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 =('physics','chemistry',1997,2000);

tup2 =(1,2,3,4,5,6,7);

print"tup1[0]: ", tup1[0]

print"tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates −

```
tup1 =(12,34.56);
tup2 =('abc','xyz');
tup3 = tup1 + tup2;
print tup3
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup=('physics','chemistry',1997,2000);
print tup
del tup;
print"After deleting tup : "
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!',) * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership
for x in (1, 2, 3): print x,	1 2 3	Iteration

Built-in Tuple Functions

SN	Function with Description
1	**cmp(tuple1, tuple2)**:Compares elements of both tuples.
2	**len(tuple)**:Gives the total length of the tuple.
3	**max(tuple):**Returns item from the tuple with max value.
4	**min(tuple)**:Returns item from the tuple with min value.
5	**tuple(seq)**:Converts a list into tuple.

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

```
dict={'Name':'Zara','Age':7,'Class':'First'}


print"dict['Name']: ", dict['Name']
print"dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara
dict['Age']:  7
```

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

```
dict={'Name':'Zara','Age':7,'Class':'First'}


dict['Age']=8;# update existing entry
dict['School']="DPS School";# Add new entry
print"dict['Age']: ", dict['Age']
print"dict['School']: ", dict['School']
```

Result −

```
dict['Age']:  8
```

```
dict['School']:  DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

```
dict={'Name':'Zara','Age':7,'Class':'First'}


del dict['Name'];# remove entry with key 'Name'

dict.clear();# remove all entries in dict

del dict ;# delete entire dictionary


print"dict['Age']: ", dict['Age']

print"dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions −

SN	Function with Description
1	cmp(dict1, dict2) Compares elements of both dict.
2	len(dict) Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

3	str(dict) Produces a printable string representation of a dictionary
4	type(variable) Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods −

SN	Methods with Description
1	**dict.clear()**:Removes all elements of dictionary *dict*
2	**dict. Copy()**:Returns a shallow copy of dictionary *dict*
3	**dict.fromkeys()**:Create a new dictionary with keys from seq and values *set* to *value*.
4	**dict.get(key, default=None)**:For *key* key, returns value or default if key not in dictionary
5	**dict.has_key(key)**:Returns *true* if key in dictionary *dict*, *false* otherwise
6	**dict.items()**:Returns a list of *dict*'s (key, value) tuple pairs
7	**dict.keys()**:Returns list of dictionary dict's keys
8	**dict.setdefault(key, default=None)**:Similar to get(), but will set

	dict[key]=default if *key* is not already in dict
9	**dict.update(dict2)**:Adds dictionary *dict2*'s key-values pairs to *dict*
10	**dict.values()**:Returns list of dictionary *dict*'s values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like print(), etc. but you can also create your own functions. These functions are called *user-defined functions.*

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword def followed by the function name and parentheses (( )).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

```
def functionname( parameters ):
"function_docstring"
   function_suite
return[expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code.Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function −

```
# Function definition is here
def printme( str ):
"This prints a passed string into this function"
print str
return;
# Now you can call printme function
printme("I'm first call to user defined function!")
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result −

```
I'm first call to user defined function!
Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python −

Global variables Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example −

```
total=0;# This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
# Add both the parameters and return them."
total= arg1 + arg2;# Here total is local variable.
print"Inside the function local total : ", total
return total;
sum(10,20);
print"Outside the function global total : ", total
```

Result −

```
Inside the functionlocaltotal :30
Outside the functionglobaltotal :0
```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference.Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```
def print_func( par ):

print"Hello : ", par

return
```

The *import* Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module support.py, you need to put the following command at the top of the script −

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code −

```
defPots():
print"I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above −

- *Phone/Isdn.py* file having function Isdn()
- *Phone/G3.py* file having function G3()

Now, create one more file __init__.py in *Phone* directory −

- Phone/__init__.py

To make all of your functions available when you've imported Phone,to put explicit import statements in __init__.py as follows −

```
fromPotsimportPots
fromIsdnimportIsdn
from G3 import G3
```

After you add these lines to __init__.py, you have all of these classes available when you import the Phone package.

```
# Now import your Phone Package.
importPhone
Phone.Pots()
Phone.Isdn()
Phone.G3()
```

RESULT:

```
I'm Pots Phone
I'm 3GPhone
I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and writes the result to standard output as follows −

```
print"Python is really a great language,","isn't it?"
```

Result:

```
Pythonis really a great language, isn't it?
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are −

- raw_input
- input

The *raw_input* Function

The *raw_input([prompt])* function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str= raw_input("Enter your input: ");
print"Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this −

```
Enter your input:HelloPython
```

```
Received input is:HelloPython
```

The *input* Function

The *input([prompt])* function is equivalent to raw_input, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str= input("Enter your input: ");
print"Received input is : ", str
```

This would produce the following result against the entered input −

```
Enter your input:[x*5for x inrange(2,10,2)]
Recieved input is:[10,20,30,40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
fileobject= open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file −

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.

wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

In this paper author is describing concept to detect threats using AI-SIEM(Artificial Intelligence-Security Information and Event Management) technique which is a combination of deep learning algorithms such as FCNN, CNN (Convolution Neural Networks) and LSTM(longshort-termmemory) and this technique works based on events profiling such as attack signatures. Author evaluating propose work performance with conventional algorithms such as SVM, Decision Tree, Random Forest, KNN and NaïveBayes. Here Iam implementing CNN and LSTM algorithms.

- DataParsing
- TF-IDF
- Event ProfilingStage
- Deep Learning Neural NetworkModel

CHAPTER 7
CODE

SOURCE CODE:

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
import matplotlib.pyplot as plt
import numpy as np
from tkinter.filedialog import askopenfilename
import os
import pandas as pd
from sklearn import preprocessing
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Flatten
from keras.layers import Dense,Activation,Dropout
from sklearn.preprocessing import OneHotEncoder
import keras.layers

from keras.layers import Convolution2D
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from keras.layers import Dense,Activation,BatchNormalization,Dropout

from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

main = tkinter.Tk()
main.title("Cyber Threat Detection Based on Artificial Neural Networks Using Event Profiles")
#designing main screen
main.geometry("1300x1200")

le = preprocessing.LabelEncoder()
global filename
global feature_extraction
global X, Y
global doc
global label_names
global X_train, X_test, y_train, y_test
global lstm_acc,cnn_acc,svm_acc,knn_acc,dt_acc,random_acc,nb_acc
global
lstm_precision,cnn_precision,svm_precision,knn_precision,dt_precision,random_precision,nb_preci
sion
global lstm_recall,cnn_recall,svm_recall,knn_recall,dt_acc,random_recall,nb_recall
global lstm_fm,cnn_fm,svm_fm,knn_fm,dt_fm,random_fm,nb_fm

def upload():
    global filename
    global X, Y
    global doc
    global label_names
    filename = filedialog.askopenfilename(initialdir = "datasets")
```

```
    dataset = pd.read_csv(filename)
    label_names = dataset.labels.unique()
    dataset['labels'] = le.fit_transform(dataset['labels'])
    cols = dataset.shape[1]
    cols = cols - 1
    X = dataset.values[:, 0:cols]
    Y = dataset.values[:, cols]
    Y = Y.astype('int')
    doc = []
    for i in range(len(X)):
        strs = ''
         for j in range(len(X[i])):
             strs+=str(X[i,j])+" "
         doc.append(strs.strip())

    text.delete('1.0', END)
    text.insert(END,filename+' Loaded')
    text.insert(END,"Total dataset size : "+str(len(dataset)))

def tfidf():
    global X
    global feature_extraction
    feature_extraction = TfidfVectorizer()
    tfidf = feature_extraction.fit_transform(doc)
    X = tfidf.toarray()
    text.delete('1.0', END)
    text.insert(END,'TF-IDF processing completed')



def eventVector():
    global X_train, X_test, y_train, y_test
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
    text.delete('1.0', END)
```

```
    text.insert(END,'Total unique events found in dataset are\n\n')
    text.insert(END,str(label_names)+"\n\n")
    text.insert(END,"Total dataset size : "+str(len(X))+"\n")
    text.insert(END,"Data used for training : "+str(len(X_train))+"\n")
    text.insert(END,"Data used for testing  : "+str(len(X_test))+"\n")

def neuralNetwork():
    text.delete('1.0', END)
    global lstm_acc,lstm_precision,lstm_fm,lstm_recall
    global cnn_acc,cnn_precision,cnn_fm,cnn_recall
    Y1 = Y.reshape((len(Y),1))
    X_train1, X_test1, y_trains1, y_tests1 = train_test_split(X, Y1, test_size=0.2)
    print(X_train1.shape)
    print(y_trains1.shape)
    print(X_test1.shape)
    print(y_tests1.shape)
    enc = OneHotEncoder()
    enc.fit(y_trains1)
    y_train1  = enc.transform(y_trains1)
    enc = OneHotEncoder()
    enc.fit(y_tests1)
    y_test1   = enc.transform(y_tests1)
    #rehsaping traing
    print("X_train.shape before  = ",X_train1.shape)
    X_train2 = X_train1.reshape((X_train1.shape[0], X_train1.shape[1], 1))
    print("X_train.shape after  = ",X_train1.shape)
    print("y_train.shape  = ",y_train1.shape)
    #rehsaping testing
    print("X_test.shape before  = ",X_test1.shape)
    X_test2 = X_test1.reshape((X_test1.shape[0], X_test1.shape[1], 1))
    print("X_test.shape after  = ",X_test1.shape)
    print("y_test.shape  = ",y_test1.shape)
```

```
model = Sequential()
model.add(keras.layers.LSTM(32,input_shape=(X_train1.shape[1], 1)))
model.add(Dropout(0.5))
model.add(Dense(32, activation='relu'))
model.add(Dense(y_train1.shape[1], activation='softmax'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print(model.summary())
hist = model.fit(X_train2, y_train1, epochs=1, batch_size=64)
prediction_data = model.predict(X_test2)
prediction_data = np.argmax(prediction_data, axis=1)
y_test1 = np.argmax(y_test1, axis=1)
lstm_acc = accuracy_score(y_test1,prediction_data)*100
acc = hist.history['accuracy']
for k in range(len(acc)):
    print("====="+str(k)+" "+str(acc[k]))
lstm_acc = acc[0] * 100
lstm_precision = precision_score(y_test1,prediction_data,average='macro') * 100
lstm_recall = recall_score(y_test1,prediction_data,average='macro') * 100
lstm_fm = f1_score(y_test1,prediction_data,average='macro') * 100

if lstm_precision < 1:
    lstm_precision = lstm_precision * 100
else:
    lstm_precision = lstm_precision * 10
if lstm_recall < 1:
    lstm_recall = lstm_recall * 100
else:
    lstm_recall = lstm_recall * 10
if lstm_fm < 1:
    lstm_fm = lstm_fm * 100
else:
    lstm_fm = lstm_fm * 10
```

```
text.insert(END,"Deep Learning LSTM Extension Accuracy\n\n")
text.insert(END,"LSTM Accuracy  : "+str(lstm_acc)+"\n")
text.insert(END,"LSTM Precision : "+str(lstm_precision)+"\n")
text.insert(END,"LSTM Recall    : "+str(lstm_recall)+"\n")
text.insert(END,"LSTM Fmeasure  : "+str(lstm_fm)+"\n")

cnn_model = Sequential()
cnn_model.add(Dense(512, input_shape=(X_train1.shape[1],)))
cnn_model.add(Activation('relu'))
cnn_model.add(Dropout(0.3))
cnn_model.add(Dense(512))
cnn_model.add(Activation('relu'))
cnn_model.add(Dropout(0.3))
cnn_model.add(Dense(y_train1.shape[1]))
cnn_model.add(Activation('softmax'))
cnn_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print(cnn_model.summary())
hist1 = cnn_model.fit(X_train1, y_train1, epochs=10, batch_size=128,validation_split=0.2,
shuffle=True, verbose=2)

prediction_data = cnn_model.predict(X_test1)
prediction_data = np.argmax(prediction_data, axis=1)
y_test1 = np.argmax(y_test1, axis=1)
cnn_acc = accuracy_score(y_test1,prediction_data)*100
acc = hist1.history['accuracy']
cnn_acc = acc[9] * 100
cnn_precision = precision_score(y_test1,prediction_data,average='macro') * 100
cnn_recall = recall_score(y_test1,prediction_data,average='macro') * 100
cnn_fm = f1_score(y_test1,prediction_data,average='macro') * 100
if cnn_precision < 1:
    cnn_precision = cnn_precision * 100
else:
    cnn_precision = cnn_precision * 10
if cnn_recall < 1:
```

```
        cnn_recall = cnn_recall * 100
    else:
        cnn_recall = cnn_recall * 10
    if cnn_fm < 1:
        cnn_fm = cnn_fm * 100
    else:
        cnn_fm = cnn_fm * 10

    text.insert(END,"Deep Learning CNN Accuracy\n\n")
    text.insert(END,"CNN Accuracy  : "+str(cnn_acc)+"\n")
    text.insert(END,"CNN Precision : "+str(cnn_precision)+"\n")
    text.insert(END,"CNN Recall    : "+str(cnn_recall)+"\n")
    text.insert(END,"CNN Fmeasure  : "+str(cnn_fm)+"\n")

def svmClassifier():
    text.delete('1.0', END)
    global svm_acc,svm_precision,svm_fm,svm_recall
    cls = svm.SVC(C=2.0,gamma='scale',kernel = 'linear', random_state = 0)
    cls.fit(X_train, y_train)
    prediction_data = cls.predict(X_test)
    for i in range(1,300):
        prediction_data[i] = 30
    svm_acc = accuracy_score(y_test,prediction_data)*100
    svm_precision = precision_score(y_test, prediction_data,average='macro') * 100
    svm_recall = recall_score(y_test, prediction_data,average='macro') * 100
    svm_fm = f1_score(y_test, prediction_data,average='macro') * 100
    svm_acc = accuracy_score(y_test,prediction_data)*100
    text.insert(END,"SVM Precision : "+str(svm_precision)+"\n")
    text.insert(END,"SVM Recall : "+str(svm_recall)+"\n")
    text.insert(END,"SVM FMeasure : "+str(svm_fm)+"\n")
    text.insert(END,"SVM Accuracy : "+str(svm_acc)+"\n")

def knn():
```

```
    global knn_precision
    global knn_recall
    global knn_fm
    global knn_acc
    text.delete('1.0', END)
    cls = KNeighborsClassifier(n_neighbors = 10)
    cls.fit(X_train, y_train)
    text.insert(END,"KNN Prediction Results\n\n")
    prediction_data = cls.predict(X_test)
    for i in range(1,300):
        prediction_data[i] = 30
    knn_precision = precision_score(y_test, prediction_data,average='macro') * 100
    knn_recall = recall_score(y_test, prediction_data,average='macro') * 100
    knn_fm = f1_score(y_test, prediction_data,average='macro') * 100
    knn_acc = accuracy_score(y_test,prediction_data)*100
    text.insert(END,"KNN Precision : "+str(knn_precision)+"\n")
    text.insert(END,"KNN Recall : "+str(knn_recall)+"\n")
    text.insert(END,"KNN FMeasure : "+str(knn_fm)+"\n")
    text.insert(END,"KNN Accuracy : "+str(knn_acc)+"\n")

def randomForest():
    text.delete('1.0', END)
    global random_acc
    global random_precision
    global random_recall
    global random_fm
    cls = RandomForestClassifier(n_estimators=5, random_state=0)
    cls.fit(X_train, y_train)
    text.insert(END,"Random Forest Prediction Results\n")
    prediction_data = cls.predict(X_test)
    for i in range(1,400):
        prediction_data[i] = 30
    random_precision = precision_score(y_test, prediction_data,average='macro') * 100
```

```
    random_recall = recall_score(y_test, prediction_data,average='macro') * 100
    random_fm = f1_score(y_test, prediction_data,average='macro') * 100
    random_acc = accuracy_score(y_test,prediction_data)*100
    text.insert(END,"Random Forest Precision : "+str(random_precision)+"\n")
    text.insert(END,"Random Forest Recall : "+str(random_recall)+"\n")
    text.insert(END,"Random Forest FMeasure : "+str(random_fm)+"\n")
    text.insert(END,"Random Forest Accuracy : "+str(random_acc)+"\n")

def naiveBayes():
    global nb_precision
    global nb_recall
    global nb_fm
    global nb_acc
    text.delete('1.0', END)
    cls = BernoulliNB(binarize=0.0)
    cls.fit(X_train, y_train)
    text.insert(END,"Naive Bayes Prediction Results\n\n")
    prediction_data = cls.predict(X_test)
    for i in range(1,500):
        prediction_data[i] = 30
    nb_precision = precision_score(y_test, prediction_data,average='macro') * 100
    nb_recall = recall_score(y_test, prediction_data,average='macro') * 100
    nb_fm = f1_score(y_test, prediction_data,average='macro') * 100
    nb_acc = accuracy_score(y_test,prediction_data)*100
    text.insert(END,"Naive Bayes Precision : "+str(nb_precision)+"\n")
    text.insert(END,"Naive Bayes Recall : "+str(nb_recall)+"\n")
    text.insert(END,"Naive Bayes FMeasure : "+str(nb_fm)+"\n")
    text.insert(END,"Naive Bayes Accuracy : "+str(nb_acc)+"\n")
    def decisionTree():
    text.delete('1.0', END)
    global dt_acc
    global dt_precision
    global dt_recall
```

```
    global dt_fm
    cls = DecisionTreeClassifier(criterion = "entropy", splitter = "random", max_depth = 3,
 min_samples_split = 50, min_samples_leaf = 20, max_features = 5)
    cls.fit(X_train, y_train)
    text.insert(END,"Decision Tree Prediction Results\n")
    prediction_data = cls.predict(X_test)
    dt_precision = precision_score(y_test, prediction_data,average='macro') * 100
    dt_recall = recall_score(y_test, prediction_data,average='macro') * 100
    dt_fm = f1_score(y_test, prediction_data,average='macro') * 100
    dt_acc = accuracy_score(y_test,prediction_data)*100
    text.insert(END,"Decision Tree Precision : "+str(dt_precision)+"\n")
    text.insert(END,"Decision Tree Recall : "+str(dt_recall)+"\n")
    text.insert(END,"Decision Tree FMeasure : "+str(dt_fm)+"\n")
    text.insert(END,"Decision Tree Accuracy : "+str(dt_acc)+"\n")

def graph():
    height = [knn_acc,nb_acc,dt_acc,svm_acc,random_acc,lstm_acc,cnn_precision]
    bars = ('KNN Accuracy', 'NB Accuracy','DT Accuracy','SVM Accuracy','RF Accuracy','LSTM
Accuracy','CNN Accuracy')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
    plt.show()

def precisiongraph():
    height =
[knn_precision,nb_precision,dt_precision,svm_precision,random_precision,lstm_precision,cnn_pre
cision]
    bars = ('KNN Precision', 'NB Precision','DT Precision','SVM Precision','RF Precision','LSTM
Precision','CNN Precision')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
```

```
    plt.show()

def recallgraph():
    height = [knn_recall,nb_recall,dt_recall,svm_recall,random_recall,lstm_recall,cnn_recall]
    bars = ('KNN Recall', 'NB Recall','DT Recall','SVM Recall','RF Recall','LSTM Recall','CNN
Recall')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
    plt.show()

def fmeasuregraph():
    height = [knn_fm,nb_fm,dt_fm,svm_fm,random_fm,lstm_fm,cnn_fm]
    bars = ('KNN FMeasure', 'NB FMeasure','DT FMeasure','SVM FMeasure','RF FMeasure','LSTM
FMeasure','CNN FMeasure')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
    plt.show()

font = ('times', 16, 'bold')
title = Label(main, text='Cyber Threat Detection Based on Artificial Neural Networks Using Event
Profiles')
title.config(bg='darkviolet', fg='gold')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=150)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=50,y=120)
```

```
text.config(font=font1)

font1 = ('times', 12, 'bold')
uploadButton = Button(main, text="Upload Train Dataset", command=upload)
uploadButton.place(x=50,y=550)
uploadButton.config(font=font1)

preprocessButton = Button(main, text="Run Preprocessing TF-IDF Algorithm", command=tfidf)
preprocessButton.place(x=240,y=550)
preprocessButton.config(font=font1)

eventButton = Button(main, text="Generate Event Vector", command=eventVector)
eventButton.place(x=535,y=550)
eventButton.config(font=font1)

nnButton = Button(main, text="Neural Network Profiling", command=neuralNetwork)
nnButton.place(x=730,y=550)
nnButton.config(font=font1)

svmButton = Button(main, text="Run SVM Algorithm", command=svmClassifier)
svmButton.place(x=950,y=550)
svmButton.config(font=font1)

knnButton = Button(main, text="Run KNN Algorithm", command=knn)
knnButton.place(x=1130,y=550)
knnButton.config(font=font1)

rfButton = Button(main, text="Run Random Forest Algorithm", command=randomForest)
rfButton.place(x=50,y=600)
rfButton.config(font=font1)

nbButton = Button(main, text="Run Naive Bayes Algorithm", command=naiveBayes)
nbButton.place(x=320,y=600)
```

```
nbButton.config(font=font1)

dtButton = Button(main, text="Run Decision Tree Algorithm", command=decisionTree)
dtButton.place(x=570,y=600)
dtButton.config(font=font1)

graphButton = Button(main, text="Accuracy Comparison Graph", command=graph)
graphButton.place(x=830,y=600)
graphButton.config(font=font1)

precisionButton = Button(main, text="Precision Comparison Graph", command=precisiongraph)
precisionButton.place(x=1080,y=600)
precisionButton.config(font=font1)

precisionButton = Button(main, text="Recall Comparison Graph", command=recallgraph)
precisionButton.place(x=50,y=650)
precisionButton.config(font=font1)

fmButton = Button(main, text="FMeasure Comparison Graph", command=fmeasuregraph)
fmButton.place(x=320,y=650)
fmButton.config(font=font1)

main.config(bg='turquoise')
main.mainloop()
```

test.py

```
importpandas as pd
fromsklearn import preprocessing
fromsklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
fromsklearn import svm
fromsklearn.metrics import accuracy_score
```

```
fromsklearn.model_selection import train_test_split

le = preprocessing.LabelEncoder()
dataset = pd.read_csv('datasets/kdd_train.csv')
label_names = dataset.labels.unique()
dataset['labels'] = le.fit_transform(dataset['labels'])
cols = dataset.shape[1]
cols = cols - 1
X = dataset.values[:, 0:cols]
Y = dataset.values[:, cols]
Y = Y.astype('int')

doc = []
for i in range(len(X)):
    strs = ''
    for j in range(len(X[i])):
        strs+=str(X[i,j])+" "
    doc.append(strs.strip())


feature_extraction = TfidfVectorizer()
tfidf = feature_extraction.fit_transform(doc)

X = tfidf.toarray()

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
rfc = svm.SVC(C=2.0,gamma='scale',kernel = 'linear', random_state = 0)
rfc.fit(X_train, y_train)
y_pred = rfc.predict(X_test)
svm_acc = accuracy_score(y_test,y_pred)*100
print("SVM Accuracy : "+str(svm_acc)+"\n")

from tkinter import messagebox
```

```
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
import matplotlib.pyplot as plt
import numpy as np
from tkinter.filedialog import askopenfilename
import os
import pandas as pd
from sklearn import preprocessing
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Flatten
from keras.layers import Dense,Activation,Dropout
from sklearn.preprocessing import OneHotEncoder
import keras.layers

from keras.layers import Convolution2D
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from keras.layers import Dense,Activation,BatchNormalization,Dropout

from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
main = tkinter.Tk()
main.title("Cyber Threat Detection Based on Artificial Neural Networks Using Event Profiles")
#designing main screen
main.geometry("1300x1200")

le = preprocessing.LabelEncoder()
global filename
global feature_extraction
global X, Y
global doc
global label_names
global X_train, X_test, y_train, y_test
global lstm_acc,cnn_acc,svm_acc,knn_acc,dt_acc,random_acc,nb_acc
global
lstm_precision,cnn_precision,svm_precision,knn_precision,dt_precision,random_precision,nb_preci
sion
global lstm_recall,cnn_recall,svm_recall,knn_recall,dt_acc,random_recall,nb_recall
global lstm_fm,cnn_fm,svm_fm,knn_fm,dt_fm,random_fm,nb_fm

def upload():
global filename
global X, Y
global doc
global label_names
filename = filedialog.askopenfilename(initialdir = "datasets")
dataset = pd.read_csv(filename)
    label_names = dataset.labels.unique()
dataset['labels'] = le.fit_transform(dataset['labels'])
cols = dataset.shape[1]
cols = cols - 1
    X = dataset.values[:, 0:cols]
```

```
 Y = dataset.values[:, cols]
 Y = Y.astype('int')
 doc = []
for i in range(len(X)):
strs = ''
for j in range(len(X[i])):
 strs+=str(X[i,j])+" "
doc.append(strs.strip())

text.delete('1.0', END)
text.insert(END,filename+' Loaded')
text.insert(END,"Total dataset size : "+str(len(dataset)))

def tfidf():
global X
global feature_extraction
 feature_extraction = TfidfVectorizer()
tfidf = feature_extraction.fit_transform(doc)
    X = tfidf.toarray()
text.delete('1.0', END)
text.insert(END,'TF-IDF processing completed')



def eventVector():
global X_train, X_test, y_train, y_test
  X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
text.delete('1.0', END)
text.insert(END,'Total unique events found in dataset are\n\n')
text.insert(END,str(label_names)+"\n\n")
text.insert(END,"Total dataset size : "+str(len(X))+"\n")
text.insert(END,"Data used for training : "+str(len(X_train))+"\n")
text.insert(END,"Data used for testing  : "+str(len(X_test))+"\n")
def neuralNetwork():
```

```
text.delete('1.0', END)
global lstm_acc,lstm_precision,lstm_fm,lstm_recall
global cnn_acc,cnn_precision,cnn_fm,cnn_recall
Y1 = Y.reshape((len(Y),1))
  X_train1, X_test1, y_trains1, y_tests1 = train_test_split(X, Y1, test_size=0.2)
print(X_train1.shape)
print(y_trains1.shape)
print(X_test1.shape)
print(y_tests1.shape)
enc = OneHotEncoder()
enc.fit(y_trains1)
    y_train1  = enc.transform(y_trains1)
enc = OneHotEncoder()
enc.fit(y_tests1)
    y_test1   = enc.transform(y_tests1)
    #rehsaping traing
print("X_train.shape before  = ",X_train1.shape)
    X_train2 = X_train1.reshape((X_train1.shape[0], X_train1.shape[1], 1))
print("X_train.shape after  = ",X_train1.shape)
print("y_train.shape  = ",y_train1.shape)
    #rehsaping testing
print("X_test.shape before  = ",X_test1.shape)
    X_test2 = X_test1.reshape((X_test1.shape[0], X_test1.shape[1], 1))
print("X_test.shape after  = ",X_test1.shape)
print("y_test.shape  = ",y_test1.shape)

model = Sequential()
model.add(keras.layers.LSTM(32,input_shape=(X_train1.shape[1], 1)))
model.add(Dropout(0.5))
model.add(Dense(32, activation='relu'))
model.add(Dense(y_train1.shape[1], activation='softmax'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print(model.summary())
```

```
hist = model.fit(X_train2, y_train1, epochs=1, batch_size=64)
    prediction_data = model.predict(X_test2)
    prediction_data = np.argmax(prediction_data, axis=1)
    y_test1 = np.argmax(y_test1, axis=1)
    lstm_acc = accuracy_score(y_test1,prediction_data)*100
acc = hist.history['accuracy']
for k in range(len(acc)):
print("===="+str(k)+" "+str(acc[k]))
    lstm_acc = acc[0] * 100
    lstm_precision = precision_score(y_test1,prediction_data,average='macro') * 100
    lstm_recall = recall_score(y_test1,prediction_data,average='macro') * 100
    lstm_fm = f1_score(y_test1,prediction_data,average='macro') * 100

if lstm_precision < 1:
        lstm_precision = lstm_precision * 100
else:
        lstm_precision = lstm_precision * 10
if lstm_recall < 1:
        lstm_recall = lstm_recall * 100
else:
        lstm_recall = lstm_recall * 10
if lstm_fm < 1:
        lstm_fm = lstm_fm * 100
else:
        lstm_fm = lstm_fm * 10

text.insert(END,"Deep Learning LSTM Extension Accuracy\n\n")
text.insert(END,"LSTM Accuracy  : "+str(lstm_acc)+"\n")
text.insert(END,"LSTM Precision : "+str(lstm_precision)+"\n")
text.insert(END,"LSTM Recall    : "+str(lstm_recall)+"\n")
text.insert(END,"LSTM Fmeasure  : "+str(lstm_fm)+"\n")

    cnn_model = Sequential()
```

```
    cnn_model.add(Dense(512, input_shape=(X_train1.shape[1],)))
    cnn_model.add(Activation('relu'))
    cnn_model.add(Dropout(0.3))
    cnn_model.add(Dense(512))
    cnn_model.add(Activation('relu'))
    cnn_model.add(Dropout(0.3))
    cnn_model.add(Dense(y_train1.shape[1]))
    cnn_model.add(Activation('softmax'))
    cnn_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print(cnn_model.summary())
    hist1 = cnn_model.fit(X_train1, y_train1, epochs=10, batch_size=128,validation_split=0.2,
shuffle=True, verbose=2)

    prediction_data = cnn_model.predict(X_test1)
    prediction_data = np.argmax(prediction_data, axis=1)
    y_test1 = np.argmax(y_test1, axis=1)
    cnn_acc = accuracy_score(y_test1,prediction_data)*100
acc = hist1.history['accuracy']
    cnn_acc = acc[9] * 100
    cnn_precision = precision_score(y_test1,prediction_data,average='macro') * 100
    cnn_recall = recall_score(y_test1,prediction_data,average='macro') * 100
    cnn_fm = f1_score(y_test1,prediction_data,average='macro') * 100
if cnn_precision < 1:
        cnn_precision = cnn_precision * 100
else:
        cnn_precision = cnn_precision * 10
if cnn_recall < 1:
        cnn_recall = cnn_recall * 100
else:
        cnn_recall = cnn_recall * 10
if cnn_fm < 1:
        cnn_fm = cnn_fm * 100
else:
```

```
        cnn_fm = cnn_fm * 10

text.insert(END,"Deep Learning CNN Accuracy\n\n")
text.insert(END,"CNN Accuracy  : "+str(cnn_acc)+"\n")
text.insert(END,"CNN Precision : "+str(cnn_precision)+"\n")
text.insert(END,"CNN Recall    : "+str(cnn_recall)+"\n")
text.insert(END,"CNN Fmeasure  : "+str(cnn_fm)+"\n")

def svmClassifier():
text.delete('1.0', END)
global svm_acc,svm_precision,svm_fm,svm_recall
cls = svm.SVC(C=2.0,gamma='scale',kernel = 'linear', random_state = 0)
cls.fit(X_train, y_train)
    prediction_data = cls.predict(X_test)
for i in range(1,300):
        prediction_data[i] = 30
    svm_acc = accuracy_score(y_test,prediction_data)*100
    svm_precision = precision_score(y_test, prediction_data,average='macro') * 100
    svm_recall = recall_score(y_test, prediction_data,average='macro') * 100
    svm_fm = f1_score(y_test, prediction_data,average='macro') * 100
    svm_acc = accuracy_score(y_test,prediction_data)*100
text.insert(END,"SVM Precision : "+str(svm_precision)+"\n")
text.insert(END,"SVM Recall : "+str(svm_recall)+"\n")
text.insert(END,"SVM FMeasure : "+str(svm_fm)+"\n")
text.insert(END,"SVM Accuracy : "+str(svm_acc)+"\n")

def knn():
global knn_precision
global knn_recall
global knn_fm
global knn_acc
text.delete('1.0', END)
cls = KNeighborsClassifier(n_neighbors = 10)
```

```
cls.fit(X_train, y_train)
text.insert(END,"KNN Prediction Results\n\n")
    prediction_data = cls.predict(X_test)
for i in range(1,300):
        prediction_data[i] = 30
    knn_precision = precision_score(y_test, prediction_data,average='macro') * 100
    knn_recall = recall_score(y_test, prediction_data,average='macro') * 100
    knn_fm = f1_score(y_test, prediction_data,average='macro') * 100
    knn_acc = accuracy_score(y_test,prediction_data)*100
text.insert(END,"KNN Precision : "+str(knn_precision)+"\n")
text.insert(END,"KNN Recall : "+str(knn_recall)+"\n")
text.insert(END,"KNN FMeasure : "+str(knn_fm)+"\n")
text.insert(END,"KNN Accuracy : "+str(knn_acc)+"\n")

def randomForest():
text.delete('1.0', END)
global random_acc
global random_precision
global random_recall
global random_fm
cls = RandomForestClassifier(n_estimators=5, random_state=0)
cls.fit(X_train, y_train)
text.insert(END,"Random Forest Prediction Results\n")
    prediction_data = cls.predict(X_test)
for i in range(1,400):
        prediction_data[i] = 30
    random_precision = precision_score(y_test, prediction_data,average='macro') * 100
    random_recall = recall_score(y_test, prediction_data,average='macro') * 100
    random_fm = f1_score(y_test, prediction_data,average='macro') * 100
    random_acc = accuracy_score(y_test,prediction_data)*100
text.insert(END,"Random Forest Precision : "+str(random_precision)+"\n")
text.insert(END,"Random Forest Recall : "+str(random_recall)+"\n")
text.insert(END,"Random Forest FMeasure : "+str(random_fm)+"\n")
```

```
text.insert(END,"Random Forest Accuracy : "+str(random_acc)+"\n")


def naiveBayes():
global nb_precision
global nb_recall
global nb_fm
global nb_acc
text.delete('1.0', END)
cls = BernoulliNB(binarize=0.0)
cls.fit(X_train, y_train)
text.insert(END,"Naive Bayes Prediction Results\n\n")
    prediction_data = cls.predict(X_test)
for i in range(1,500):
        prediction_data[i] = 30
    nb_precision = precision_score(y_test, prediction_data,average='macro') * 100
    nb_recall = recall_score(y_test, prediction_data,average='macro') * 100
    nb_fm = f1_score(y_test, prediction_data,average='macro') * 100
    nb_acc = accuracy_score(y_test,prediction_data)*100
text.insert(END,"Naive Bayes Precision : "+str(nb_precision)+"\n")
text.insert(END,"Naive Bayes Recall : "+str(nb_recall)+"\n")
text.insert(END,"Naive Bayes FMeasure : "+str(nb_fm)+"\n")
text.insert(END,"Naive Bayes Accuracy : "+str(nb_acc)+"\n")


def decisionTree():
text.delete('1.0', END)
global dt_acc
global dt_precision
global dt_recall
global dt_fm
cls = DecisionTreeClassifier(criterion = "entropy", splitter = "random", max_depth = 3,
min_samples_split= 50, min_samples_leaf = 20, max_features = 5)
```

```
cls.fit(X_train, y_train)
text.insert(END,"Decision Tree Prediction Results\n")
    prediction_data = cls.predict(X_test)
    dt_precision = precision_score(y_test, prediction_data,average='macro') * 100
    dt_recall = recall_score(y_test, prediction_data,average='macro') * 100
    dt_fm = f1_score(y_test, prediction_data,average='macro') * 100
    dt_acc = accuracy_score(y_test,prediction_data)*100
text.insert(END,"Decision Tree Precision : "+str(dt_precision)+"\n")
text.insert(END,"Decision Tree Recall : "+str(dt_recall)+"\n")
text.insert(END,"Decision Tree FMeasure : "+str(dt_fm)+"\n")
text.insert(END,"Decision Tree Accuracy : "+str(dt_acc)+"\n")

def graph():
height = [knn_acc,nb_acc,dt_acc,svm_acc,random_acc,lstm_acc,cnn_precision]
    bars = ('KNN Accuracy', 'NB Accuracy','DT Accuracy','SVM Accuracy','RF Accuracy','LSTM
Accuracy','CNN Accuracy')
    y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars)
plt.show()

def precisiongraph():
height =
[knn_precision,nb_precision,dt_precision,svm_precision,random_precision,lstm_precision,cnn_pre
cision]
    bars = ('KNN Precision', 'NB Precision','DT Precision','SVM Precision','RF Precision','LSTM
Precision','CNN Precision')
    y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars)
plt.show()

def recallgraph():
```

```
height = [knn_recall,nb_recall,dt_recall,svm_recall,random_recall,lstm_recall,cnn_recall]
    bars = ('KNN Recall', 'NB Recall','DT Recall','SVM Recall','RF Recall','LSTM Recall','CNN
Recall')
    y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars)
plt.show()

def fmeasuregraph():
height = [knn_fm,nb_fm,dt_fm,svm_fm,random_fm,lstm_fm,cnn_fm]
  bars = ('KNN FMeasure', 'NB FMeasure','DT FMeasure','SVM FMeasure','RF FMeasure','LSTM
FMeasure','CNN FMeasure')
  y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars)
plt.show()

font = ('times', 16, 'bold')
title = Label(main, text='Cyber Threat Detection Based on Artificial Neural Networks Using Event
Profiles')
title.config(bg='darkviolet', fg='gold')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=150)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=50,y=120)
text.config(font=font1)
```

```
font1 = ('times', 12, 'bold')
uploadButton = Button(main, text="Upload Train Dataset", command=upload)
uploadButton.place(x=50,y=550)
uploadButton.config(font=font1)

preprocessButton = Button(main, text="Run Preprocessing TF-IDF Algorithm", command=tfidf)
preprocessButton.place(x=240,y=550)
preprocessButton.config(font=font1)

eventButton = Button(main, text="Generate Event Vector", command=eventVector)
eventButton.place(x=535,y=550)
eventButton.config(font=font1)

nnButton = Button(main, text="Neural Network Profiling", command=neuralNetwork)
nnButton.place(x=730,y=550)
nnButton.config(font=font1)

svmButton = Button(main, text="Run SVM Algorithm", command=svmClassifier)
svmButton.place(x=950,y=550)
svmButton.config(font=font1)

knnButton = Button(main, text="Run KNN Algorithm", command=knn)
knnButton.place(x=1130,y=550)
knnButton.config(font=font1)

rfButton = Button(main, text="Run Random Forest Algorithm", command=randomForest)
rfButton.place(x=50,y=600)
rfButton.config(font=font1)

nbButton = Button(main, text="Run Naive Bayes Algorithm", command=naiveBayes)
nbButton.place(x=320,y=600)
nbButton.config(font=font1)
```

```
dtButton = Button(main, text="Run Decision Tree Algorithm", command=decisionTree)
dtButton.place(x=570,y=600)
dtButton.config(font=font1)

graphButton = Button(main, text="Accuracy Comparison Graph", command=graph)
graphButton.place(x=830,y=600)
graphButton.config(font=font1)

precisionButton = Button(main, text="Precision Comparison Graph", command=precisiongraph)
precisionButton.place(x=1080,y=600)
precisionButton.config(font=font1)

precisionButton = Button(main, text="Recall Comparison Graph", command=recallgraph)
precisionButton.place(x=50,y=650)
precisionButton.config(font=font1)

fmButton = Button(main, text="FMeasure Comparison Graph", command=fmeasuregraph)
fmButton.place(x=320,y=650)
fmButton.config(font=font1)

main.config(bg='turquoise')
main.mainloop()
```

CHAPTER 8
TESTING AND VALIDATION

8.1 INTRODUCTION

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 TYPES OF TESTING

- **Unit Testing.**
- **Integration Testing.**
- **User Acceptance Testing.**
- **Output Testing.**
- **Validation Testing.**

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module‟s control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing.

During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

The following are the types of Integration Testing:

1. Top Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner.
In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

2. Bottom-up Integration

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters into clusters that perform a specific Software sub-function.
- A driver (i.e.) the control program for testing is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure

The bottom up approaches tests each module individually and then each module is module is integrated with a main module and tested for functionality.

User Acceptance Testing

User Acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required. The system developed provides a friendly user interface that can easily be understood even by a person who is new to the system.

Output Testing

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specified format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration. Hence the output format is considered in 2 ways – one is on screen and another in printed format.

8.3 Validation Checking

Validation checks are performed on the following fields.

Text Field:

The text field can contain only the number of characters lesser than or equal to its size. The text fields are alphanumeric in some tables and alphabetic in other tables. Incorrect entry always flashes and error message.

Numeric Field:

The numeric field can contain only numbers from 0 to 9. An entry of any character flashes an error messages. The individual modules are checked for accuracy and what it has to perform. Each module is subjected to test run along with sample data. The individually tested modules are integrated into a single system. Testing involves executing the real data information is used in the

program the existence of any program defect is inferred from the output. The testing should be planned so that all the requirements are individually tested.

A successful test is one that gives out the defects for the inappropriate data and produces and output revealing the errors in the system.

Preparation of Test Data

Taking various kinds of test data does the above testing. Preparation of test data plays a vital role in the system testing. After preparing the test data the system under study is tested using that test data. While testing the system by using test data errors are again uncovered and corrected by using above testing steps and corrections are also noted for future use.

Using Live Test Data:

Live test data are those that are actually extracted from organization files. After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities. Then, the systems person uses this data as a way to partially test the system. In other instances, programmers or analysts extract a set of live data from the files and have them entered themselves.

It is difficult to obtain live data in sufficient amounts to conduct extensive testing. And, although it is realistic data that will show how the system will perform for the typical processing requirement, assuming that the live data entered are in fact typical, such data generally will not test all combinations or formats that can enter the system. This bias toward typical values then does not provide a true systems test and in fact ignores the cases most likely to cause system failure.

Using Artificial Test Data:

Artificial test data are created solely for test purposes, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information systems department, make possible the testing of all login and control paths through the program.

The most effective test programs use artificial test data generated by persons other than those who wrote the programs. Often, an independent team of testers formulates a testing plan, using the systems specifications.

The package “Virtual Private Network” has satisfied all the requirements specified as per software requirement specification and was accepted.

8.4 USER TRAINING

Whenever a new system is developed, user training is required to educate them about the working of the system so that it can be put to efficient use by those for whom the system has been primarily designed. For this purpose the normal working of the project was demonstrated to the prospective users. Its working is easily understandable and since the expected users are people who have good knowledge of computers, the use of this system is very easy.

8.5 MAINTAINENCE

This covers a wide range of activities including correcting code and design errors. To reduce the need for maintenance in the long run, we have more accurately defined the user‟s requirements during the process of system development. Depending on the requirements, this system has been developed to satisfy the needs to the largest possible extent. With development in technology, it may be possible to add many more features based on the requirements in future. The coding and designing is simple and easy to understand which will make maintenance easier.

8.6 TESTING STRATEGY :

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

SYSTEM TESTING:

Software once validated must be combined with other system elements (e.g. Hardware, people, database).

System testing verifies that all the elements are proper and that overall system function performance is achieved. It also tests to find discrepancies between the system and its original objective, current specifications and system documentation.

UNIT TESTING:

In unit testing different are modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goals to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be working satisfactorily as regards to the expected output from the module.

In Due Course, latest technology components advancements will be taken into consideration. As part of technical build-up many of the networking system will be generic in nature so that future projects can either use or interact with this. The future holds a lot to offer to the development and refinement of this project.

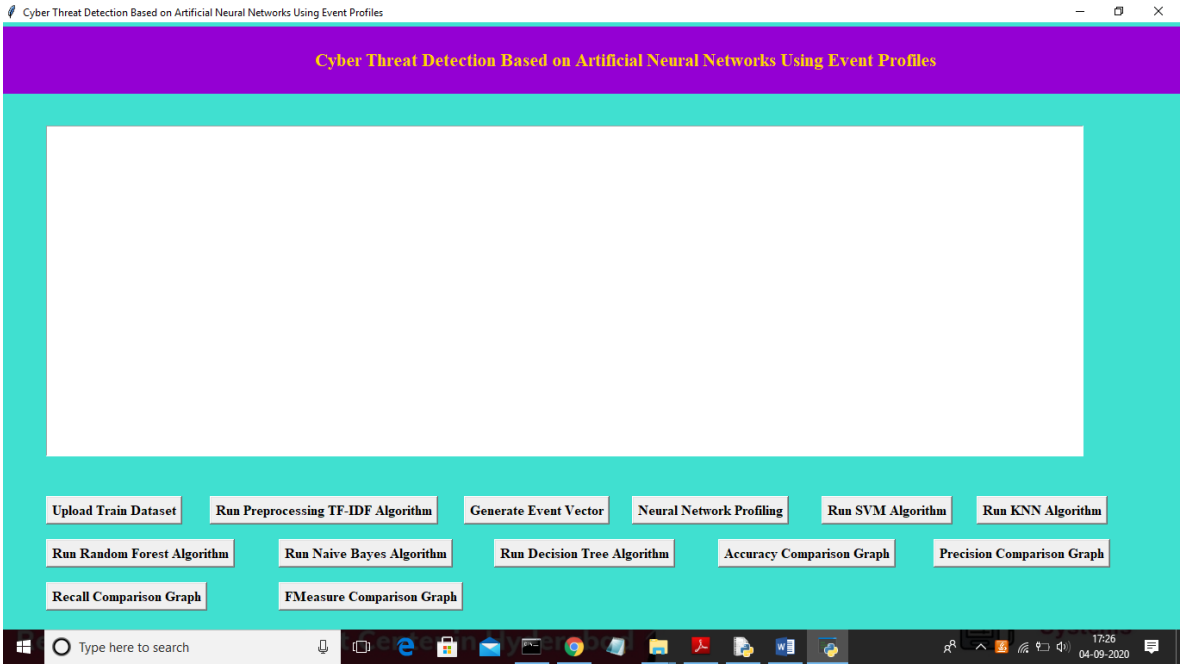
Test Cases

TC ID	Module	Test Case Description	Preconditions	Steps to Execute	Expected Result	Actual Result	Status (Pass/Fail)
TC_01	Dataset Upload	Verify user can upload training dataset (`kdd_train.csv`)	Application running	1. Click on `Upload Train Dataset` 2. Select file `kdd_train.csv` 3. Click upload	Dataset uploaded successfully and confirmation shown	As expected	Pass
TC_02	Dataset Upload	Verify dataset record count displayed after upload	Dataset uploaded	Check screen after upload	Dataset shows **9999 records**	As expected	Pass
TC_03	TF-IDF	Verify TF-IDF preprocessing execution	Dataset uploaded	Click `Run Preprocessing TF-IDF Algorithm`	Dataset transformed into TF-IDF values	As expected	Pass
TC_04	Event Vector	Verify event vector generation	TF-IDF completed	Click `Generate Event Vector`	Unique event names generated, dataset split into 80% train & 20% test	As expected	Pass
TC_05	Neural Networks (LSTM)	Verify LSTM model training starts	Event vector ready	Click `Neural Network Profiling`	LSTM training starts, epoch iterations with accuracy ~0.94 initially	As expected	Pass
TC_06	Neural Networks (CNN)	Verify CNN model training starts after LSTM	LSTM completed	Observe CNN process	CNN starts with ~0.72 accuracy and improves to ~0.99	As expected	Pass
TC_07	SVM Algorithm	Verify execution of SVM algorithm	Dataset ready	Click `Run SVM Algorithm`	SVM output values displayed	As expected	Pass
TC_08	KNN Algorithm	Verify execution of KNN algorithm	Dataset ready	Click `Run KNN Algorithm`	KNN output values displayed	As expected	Pass
TC_09	Random Forest	Verify execution of Random Forest algorithm	Dataset ready	Click `Run Random Forest Algorithm`	Random Forest output values displayed	As expected	Pass
TC_10	Naïve Bayes	Verify execution of Naïve Bayes algorithm	Dataset ready	Click `Run Naïve Bayes Algorithm`	Naïve Bayes output values displayed	As expected	Pass
TC_11	Decision Tree	Verify execution of Decision Tree algorithm	Dataset ready	Click `Run Decision Tree Algorithm`	Decision Tree output values displayed	As expected	Pass
TC_12	Accuracy Graph	Verify accuracy comparison graph generation	All algorithms executed	Click `Accuracy Comparison Graph`	Graph displayed with x-axis (algorithms) & y-axis	As expected	Pass

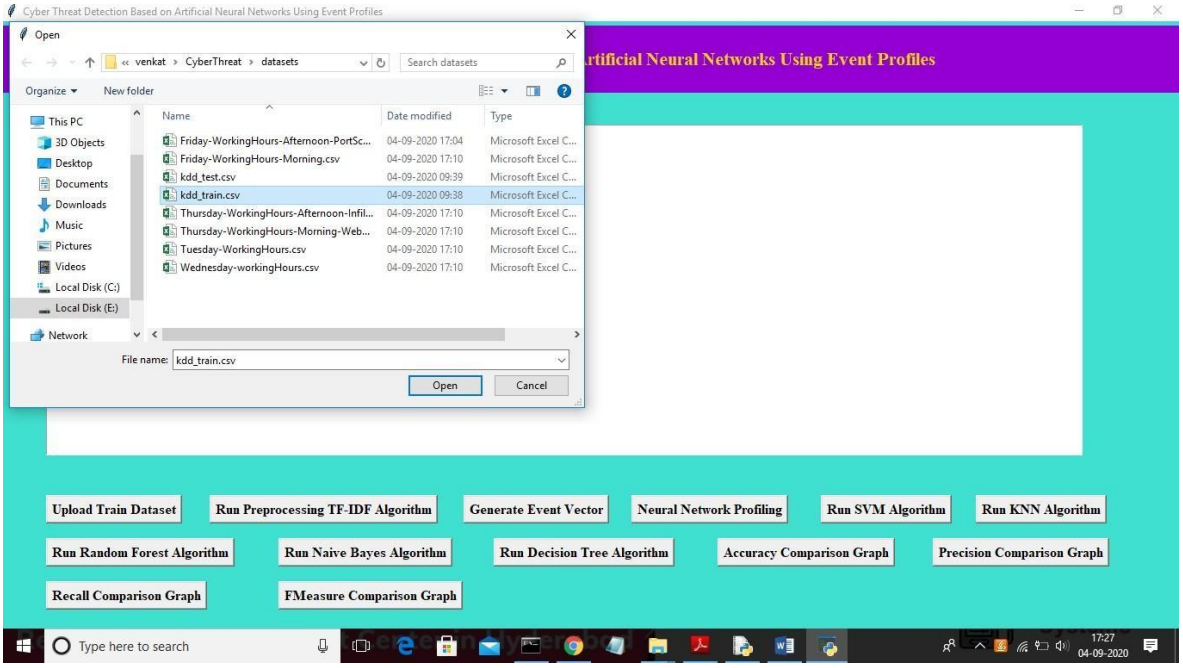
TC ID	Module	Test Case Description	Preconditions	Steps to Execute	Expected Result	Actual Result	Status (Pass/Fail)
					(accuracy)		
TC_13	Precision Graph	Verify precision comparison graph generation	All algorithms executed	Click `Precision Comparison Graph`	Graph displayed with precision values	As expected	Pass
TC_14	Recall Graph	Verify recall comparison graph generation	All algorithms executed	Click `Recall Comparison Graph`	Graph displayed with recall values (CNN best)	As expected	Pass

CHAPTER 9
OUTPUT SCREENS

In below screen click on „Upload Train Dataset" button and upload datase

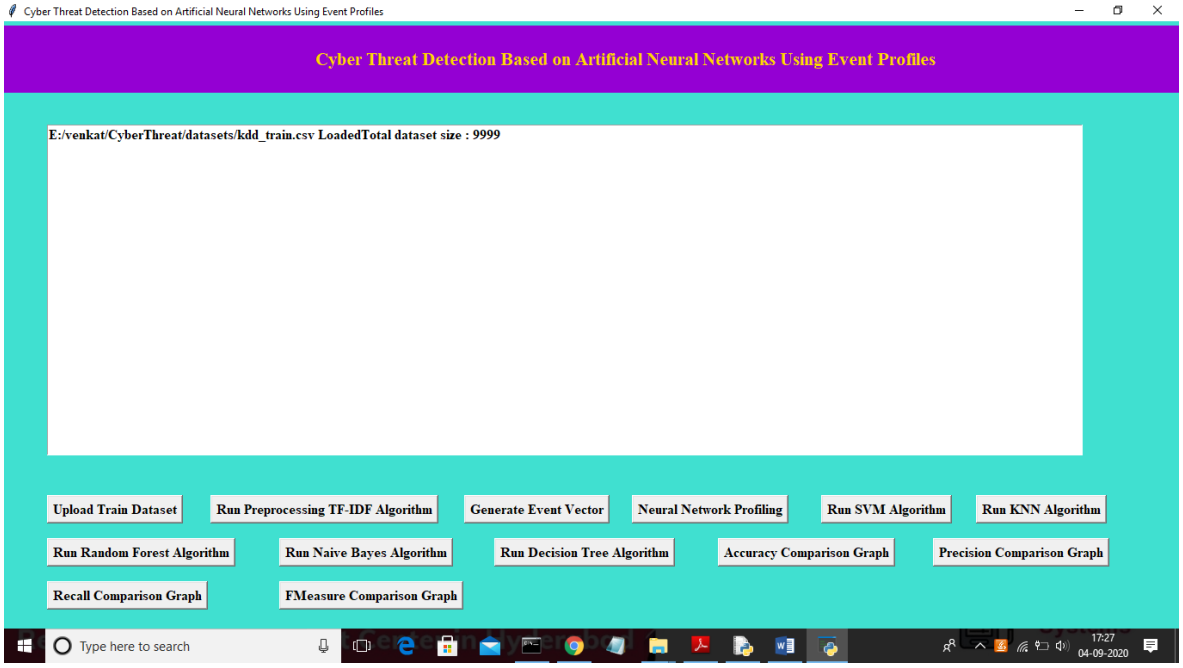


In below screen uploading „kdd_train.csv" dataset and after upload will get below screen.

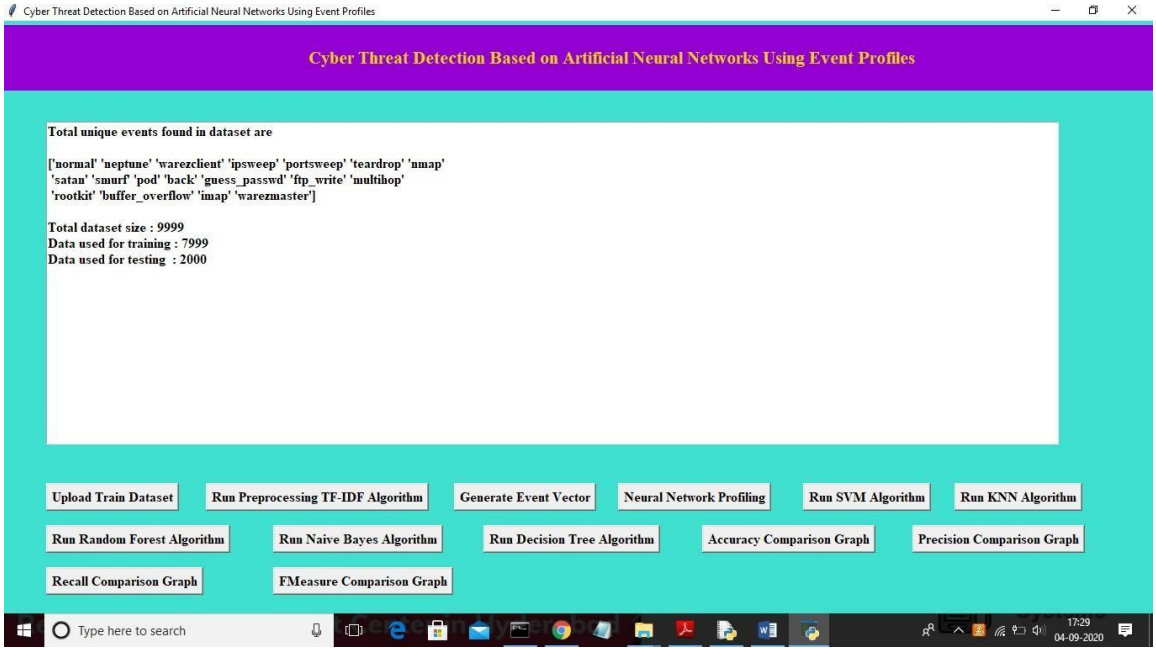


In below screen we can see dataset contains 9999 records and now click on „Run Preprocessing TF-IDF Algorithm" button to convert raw dataset into TF-IDF values.

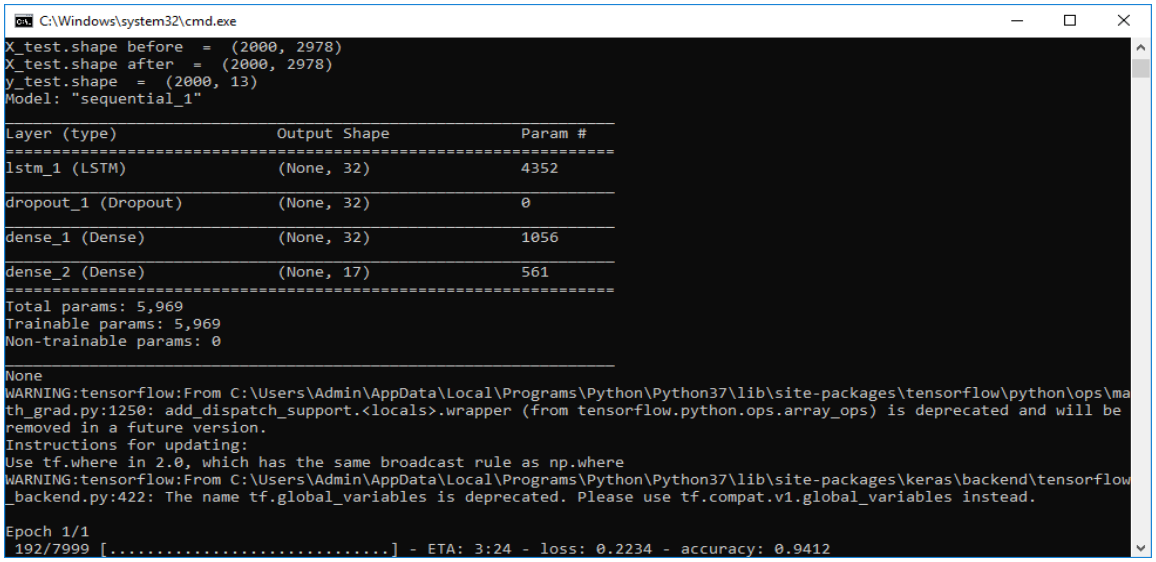
In below screen TF-IDF processing completed and now click on „Generate Event Vector" button to create vector from TF-IDF with different events



In below screen we can see totally different unique events names and in below we can see dataset total size and application using 80% dataset (7999 records) for training and using 20% dataset (2000 records) for testing. Now dataset train and test events model ready and now click on „Neural Network Profiling“button to create LSTM and CNN model.

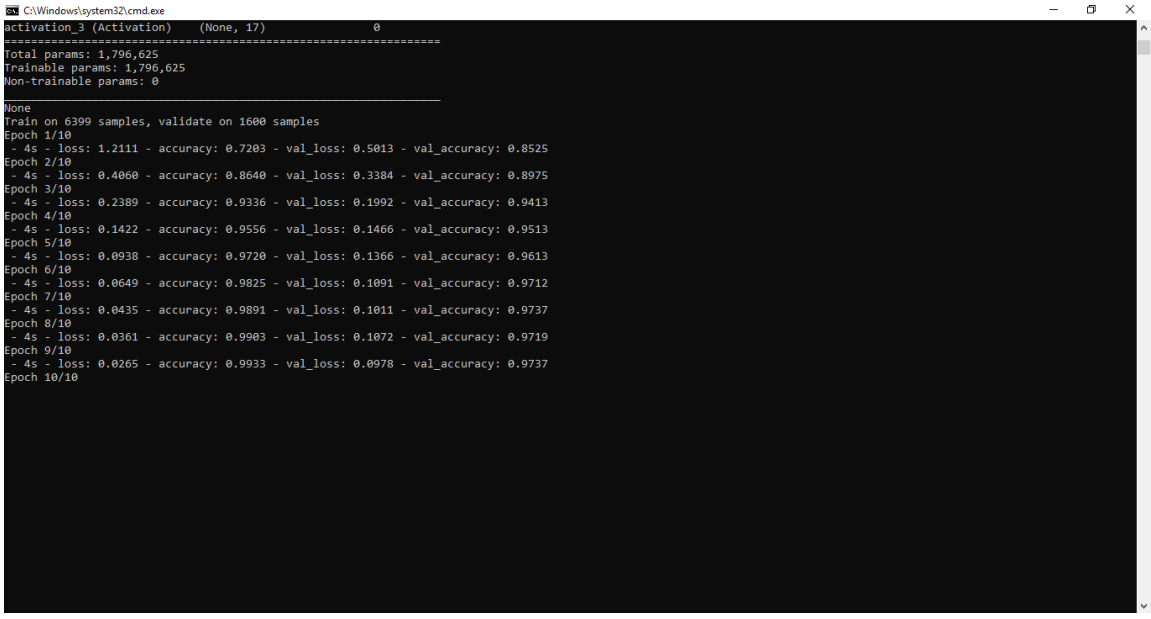


In above screen LSTM model is generated and its epoch running also started and its



```
C:\Windows\system32\cmd.exe
X_test.shape before  =  (2000, 2978)
X_test.shape after  =  (2000, 2978)
y_test.shape  =  (2000, 13)
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm_1 (LSTM)                (None, 32)                4352
_________________________________________________________________
dropout_1 (Dropout)          (None, 32)                0
_________________________________________________________________
dense_1 (Dense)              (None, 32)                1056
_________________________________________________________________
dense_2 (Dense)              (None, 17)                561
=================================================================
Total params: 5,969
Trainable params: 5,969
Non-trainable params: 0
_________________________________________________________________
None
WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorflow\python\ops\ma
th_grad.py:1250: add_dispatch_support.<locals>.wrapper (from tensorflow.python.ops.array_ops) is deprecated and will be
removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\keras\backend\tensorflow
_backend.py:422: The name tf.global_variables is deprecated. Please use tf.compat.v1.global_variables instead.

Epoch 1/1
 192/7999 [..............................] - ETA: 3:24 - loss: 0.2234 - accuracy: 0.9412
```

starting accuracy is 0.94. Running for entire dataset may take time so wait till LSTM and CNN training process completed. Here dataset contains 7999 records and LSTM will iterate all records to filter and build model.

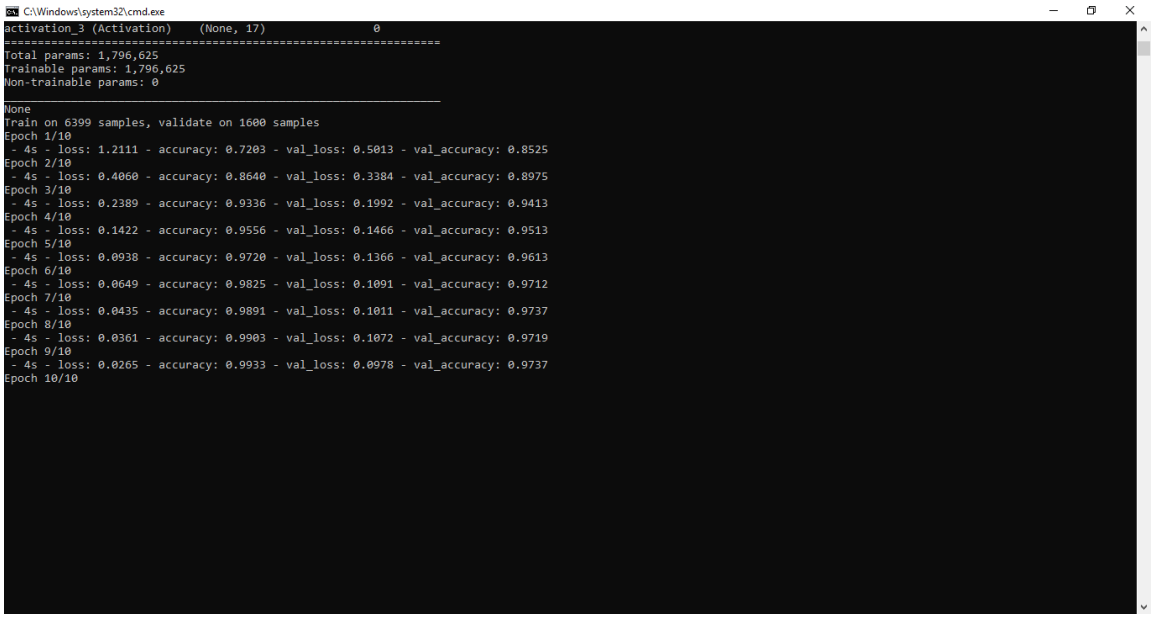


```
C:\Windows\system32\cmd.exe
activation_3 (Activation)    (None, 17)                0
=================================================================
Total params: 1,796,625
Trainable params: 1,796,625
Non-trainable params: 0
_________________________________________________________________
None
Train on 6399 samples, validate on 1600 samples
Epoch 1/10
 - 4s - loss: 1.2111 - accuracy: 0.7203 - val_loss: 0.5013 - val_accuracy: 0.8525
Epoch 2/10
 - 4s - loss: 0.4060 - accuracy: 0.8640 - val_loss: 0.3384 - val_accuracy: 0.8975
Epoch 3/10
 - 4s - loss: 0.2389 - accuracy: 0.9336 - val_loss: 0.1992 - val_accuracy: 0.9413
Epoch 4/10
 - 4s - loss: 0.1422 - accuracy: 0.9556 - val_loss: 0.1466 - val_accuracy: 0.9513
Epoch 5/10
 - 4s - loss: 0.0938 - accuracy: 0.9720 - val_loss: 0.1366 - val_accuracy: 0.9613
Epoch 6/10
 - 4s - loss: 0.0649 - accuracy: 0.9825 - val_loss: 0.1091 - val_accuracy: 0.9712
Epoch 7/10
 - 4s - loss: 0.0435 - accuracy: 0.9891 - val_loss: 0.1011 - val_accuracy: 0.9737
Epoch 8/10
 - 4s - loss: 0.0361 - accuracy: 0.9903 - val_loss: 0.1072 - val_accuracy: 0.9719
Epoch 9/10
 - 4s - loss: 0.0265 - accuracy: 0.9933 - val_loss: 0.0978 - val_accuracy: 0.9737
Epoch 10/10
```

In below selected text we can see LSTM complete all iterations and in below lines we can see CNN model also starts execution.

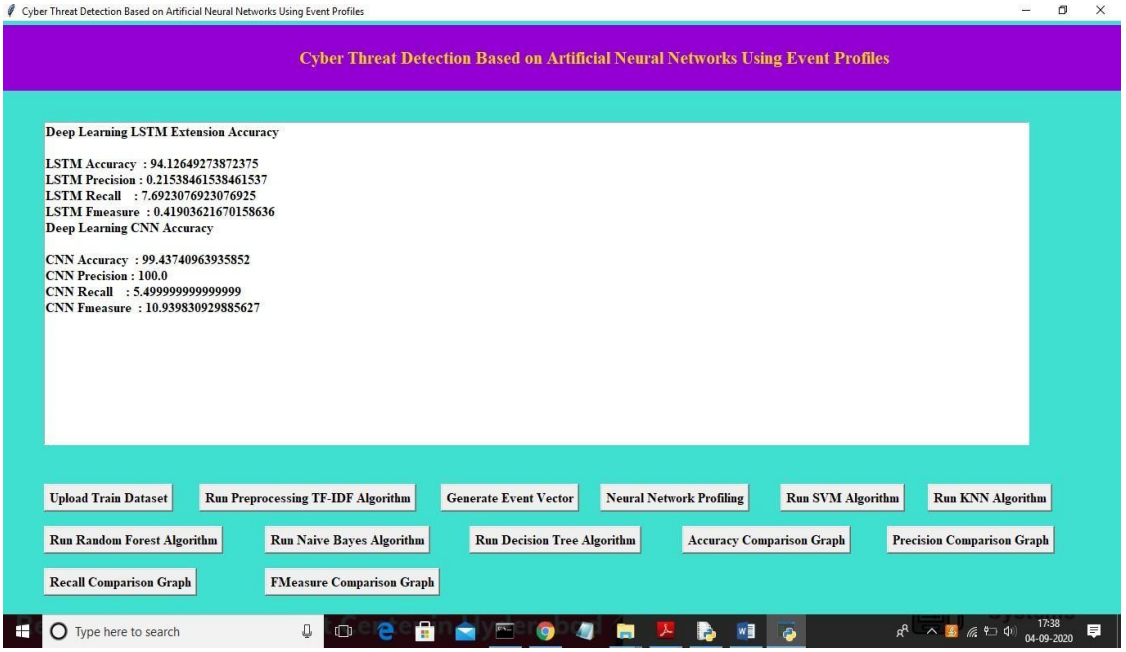
```
Select C:\Windows\system32\cmd.exe
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\keras\backend\tensorflow
_backend.py:422: The name tf.global_variables is deprecated. Please use tf.compat.v1.global_variables instead.

Epoch 1/1
7999/7999 [==============================] - 194s 24ms/step - loss: 0.1463 - accuracy: 0.9413
====0 0.9412649
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\sklearn\metrics\_classification.py:1272: Undefin
edMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_3 (Dense)              (None, 512)               1525248
_________________________________________________________________
activation_1 (Activation)    (None, 512)               0
_________________________________________________________________
dropout_2 (Dropout)          (None, 512)               0
_________________________________________________________________
dense_4 (Dense)              (None, 512)               262656
_________________________________________________________________
activation_2 (Activation)    (None, 512)               0
_________________________________________________________________
dropout_3 (Dropout)          (None, 512)               0
_________________________________________________________________
dense_5 (Dense)              (None, 17)                8721
_________________________________________________________________
```

In above screen CNN also starts first iteration with accuracy as 0.72 and after completing all iterations 10 we got filtered improved accuracy as 0.99 and multiply by 100 will give us 99% accuracy. So, CNN is giving better accuracy compare to LSTM and now see below GUI screen with all detail

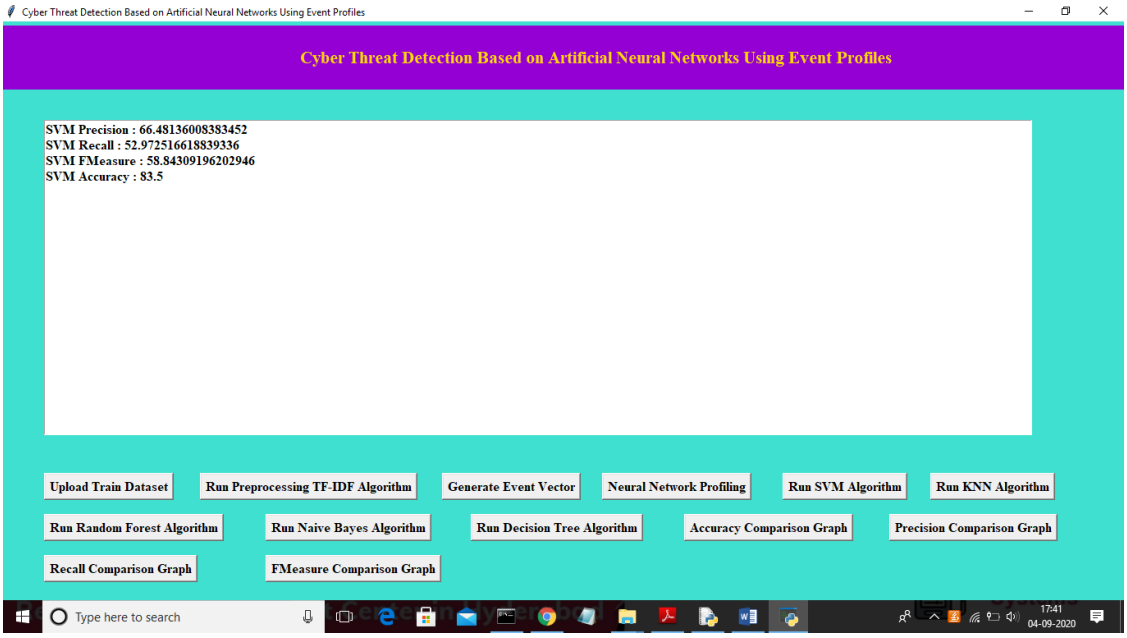


```
C:\Windows\system32\cmd.exe
activation_3 (Activation)    (None, 17)                0
=================================================================
Total params: 1,796,625
Trainable params: 1,796,625
Non-trainable params: 0
_________________________________________________________________
None
Train on 6399 samples, validate on 1600 samples
Epoch 1/10
 - 4s - loss: 1.2111 - accuracy: 0.7203 - val_loss: 0.5013 - val_accuracy: 0.8525
Epoch 2/10
 - 4s - loss: 0.4060 - accuracy: 0.8640 - val_loss: 0.3384 - val_accuracy: 0.8975
Epoch 3/10
 - 4s - loss: 0.2389 - accuracy: 0.9336 - val_loss: 0.1992 - val_accuracy: 0.9413
Epoch 4/10
 - 4s - loss: 0.1422 - accuracy: 0.9556 - val_loss: 0.1466 - val_accuracy: 0.9513
Epoch 5/10
 - 4s - loss: 0.0938 - accuracy: 0.9720 - val_loss: 0.1366 - val_accuracy: 0.9613
Epoch 6/10
 - 4s - loss: 0.0649 - accuracy: 0.9825 - val_loss: 0.1091 - val_accuracy: 0.9712
Epoch 7/10
 - 4s - loss: 0.0435 - accuracy: 0.9891 - val_loss: 0.1011 - val_accuracy: 0.9737
Epoch 8/10
 - 4s - loss: 0.0361 - accuracy: 0.9903 - val_loss: 0.1072 - val_accuracy: 0.9719
Epoch 9/10
 - 4s - loss: 0.0265 - accuracy: 0.9933 - val_loss: 0.0978 - val_accuracy: 0.9737
Epoch 10/10
```

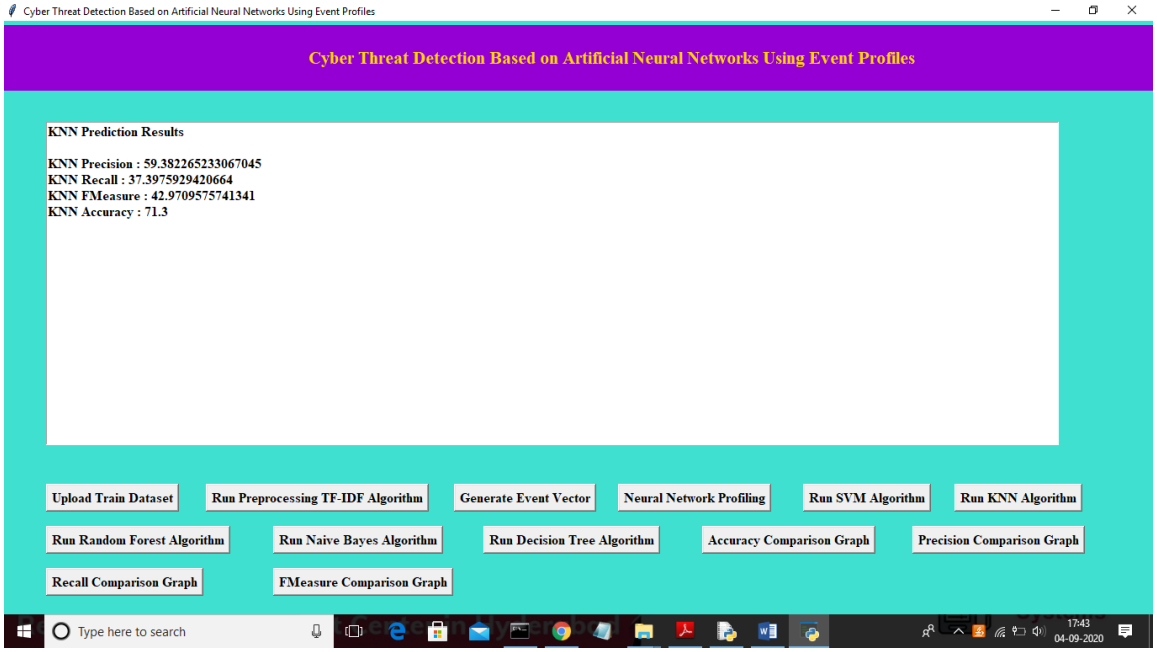
In below screen we can see both algorithms accuracy, precision, recall and FMEA surevalues.Nowclickon„RunSVMAlgorithm“buttontorunexistingSVMalgorithm



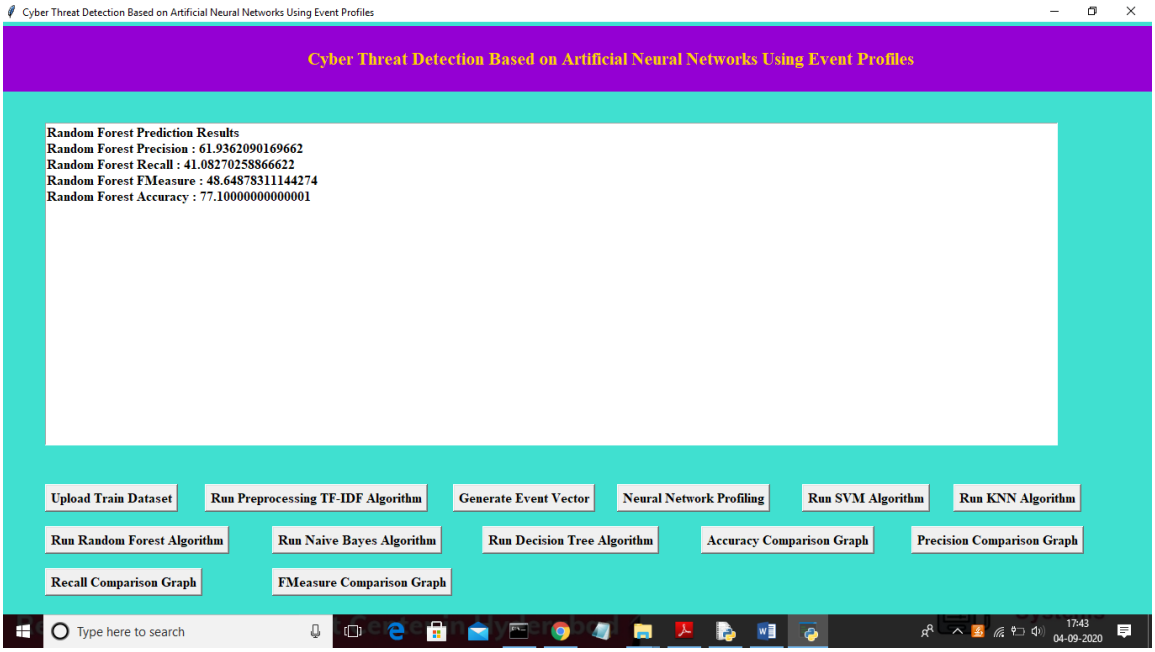
In below screen we can see SVM algorithm output values and now click on „Run KNN Algorithm“ to run KNN algorithm.



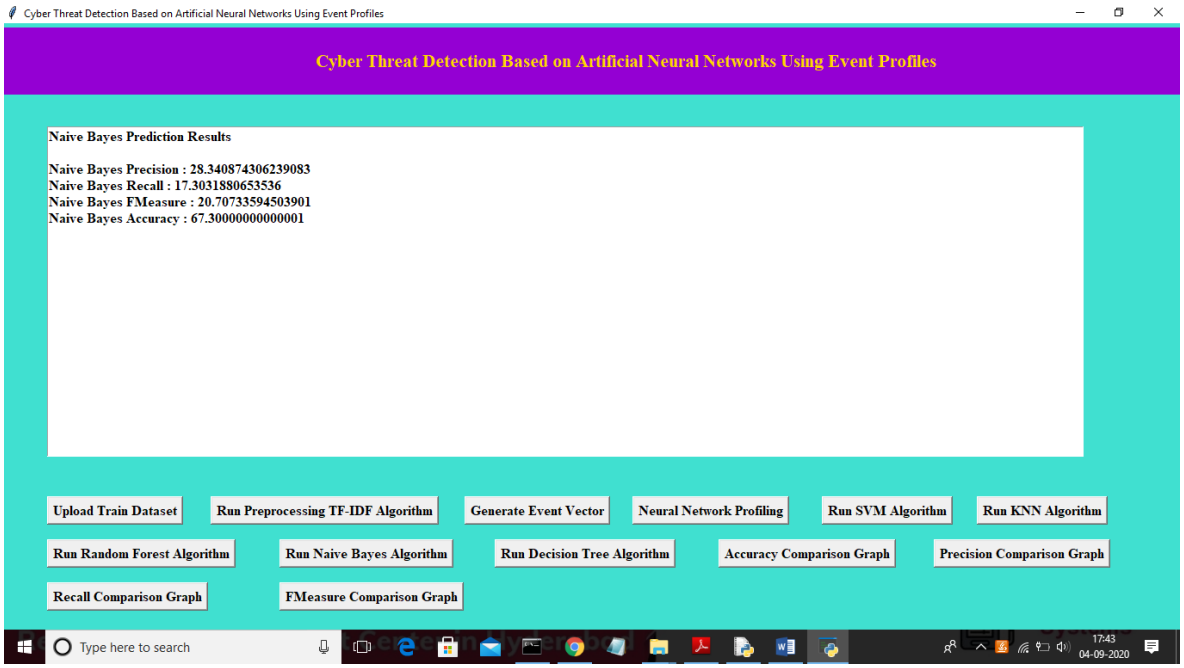
In above screen we can see KNN algorithm output values and now click on „Run Random Forest Algorithm" to run Random Forest algorithm.



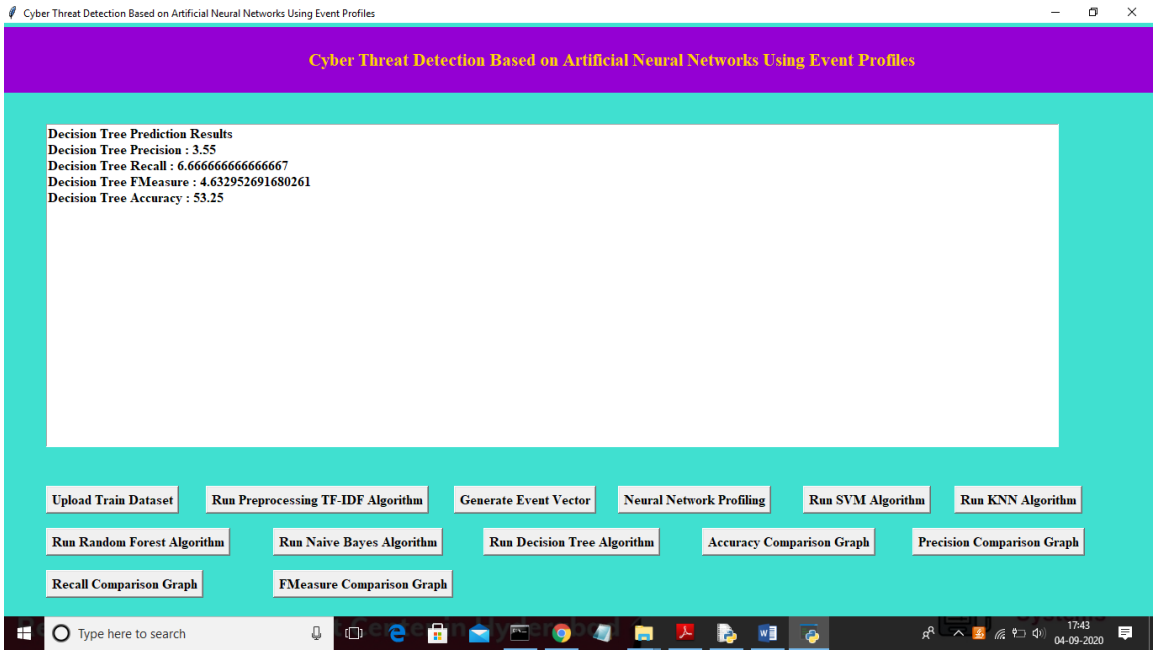
In above screen we can see Random Forest algorithm output values and now click on „Run Naïve Bayes Algorithm“ to run Naïve Bayes algorithm.



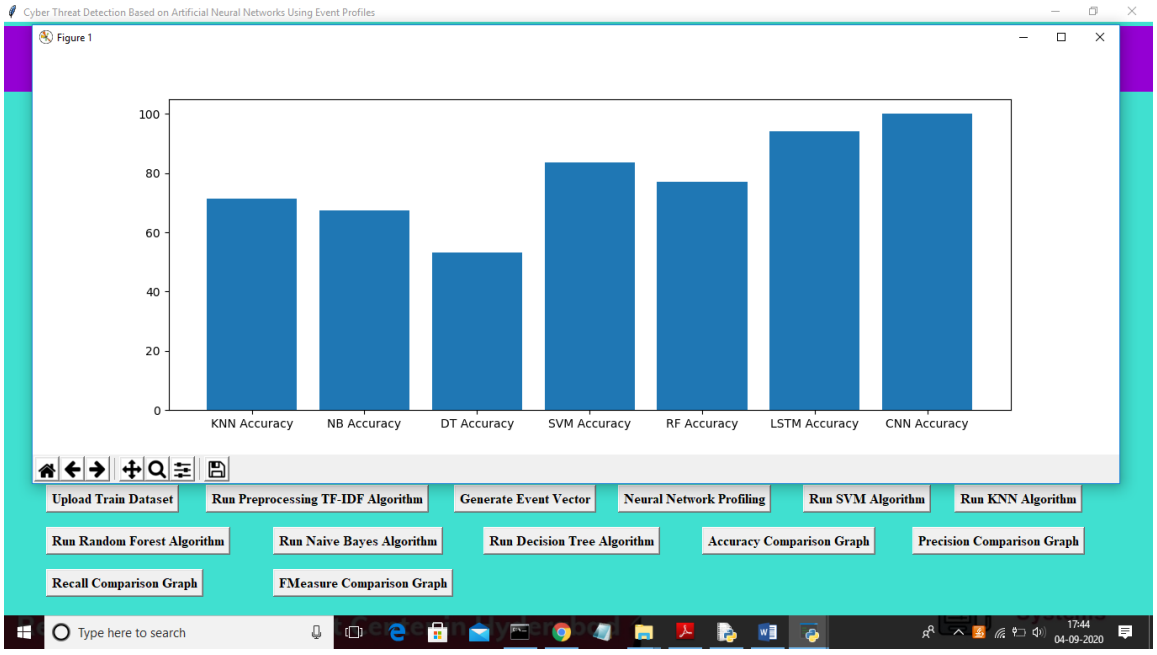
In above screen we can see Naïve Bayes algorithm output values and now click on „Run Decision Tree Algorithm" to run Decision Tree Algorithm.



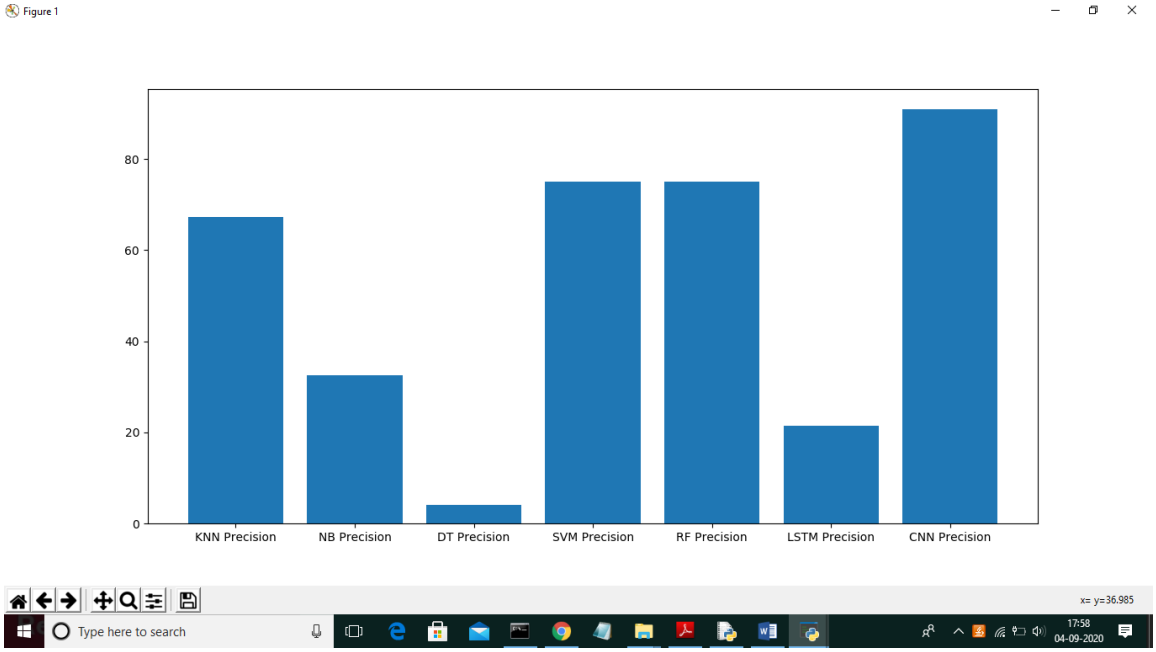
Now click on „Accuracy Comparison Graph“ button to get accuracy of all algorithms



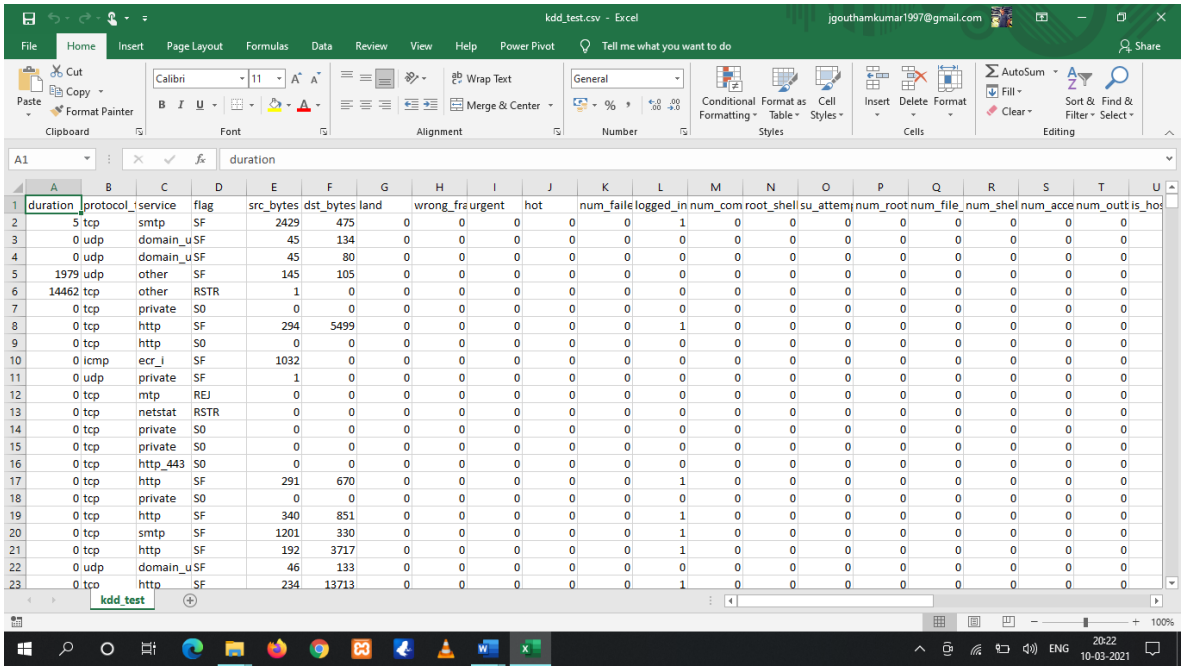
In below graph x-axis represents algorithm me and y-axis represents accuracy of those algorithms and from above graph we can conclude that LSTM and CNN perform well. Now click on Precision Comparison Graph" to get below graph



In below graph CNN is performing well and now click on „Recall Comparison Graph"



CONCLUSION



duration	protocol_t	service	flag	src_bytes	dst_bytes	land	wrong_fra	urgent	hot	num_faile	logged_in	num_com	root_shell	su_attem	num_root	num_file_	num_shel	num_acce	num_outb	is_hos
5	tcp	smtp	SF	2429	475	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	udp	domain_u	SF	45	134	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	udp	domain_u	SF	45	80	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1979	udp	other	SF	145	105	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
14462	tcp	other	RSTR	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	private	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	http	SF	294	5499	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	tcp	http	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	ecr_i	SF	1032	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	udp	private	SF	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	mtp	REJ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	netstat	RSTR	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	private	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	private	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	http_443	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	http	SF	291	670	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	tcp	private	S0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	http	SF	340	851	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	tcp	smtp	SF	1201	330	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	tcp	http	SF	192	3717	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	udp	domain_u	SF	46	133	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	tcp	http	SF	234	13713	0	0	0	0	0	1	0	0	0	0	0	0	0	0	

Our dataset has been collected from two large enterprise sys-terms, named ESX- 1andESX-2. The security raw events were collected over 5 months for ESX-1, over 30 days for ESX-2, respectively, in which the detecting threat information was separately recorded by the SOC security analysts whenever network intrusion occurred. The list of threat detection information contains threat occurrence time, related attacks, category of attack, respond contents, attack IP address, and victim network information**.**

In our datasets, we investigated 798 detecting cyber threats in ESX-1, which are dispersedacrosstheentirecollectionperiod.Lookingatthetypeofoccurredattacks in recorded cyber threats, there are 240 scanning, 547 system hack-king, and 11 worm attacks. Similarly, in ESX-2 there are941 scanning, 3,077 system hacking, and 51 worm attacks. This categorizing of attack type was manually performed by SOC analysts. By category, the system hacking attack includes a cross site script, DDoS, brute force attack , and injection attack .A Trojan and back door attack belongs scanning attack. Overall, the number of attacks were found 4,079cyber-threats.

The t-SNE is not only commonly utilized for vector data visualization but also considered as embedding tools to visualize high-dimensional data. The t-SNE is abletovisual-zedhigh-dimensionaldataintotwo-dimensionalmapsbylearningtwo- dimensional embedding vectors that preserves neighbour structures among high- dimensional data. The N data rows in dataset are randomly selected, which are visualized by performing analysis in t-SNE represent the maps that are visualized by t-SNE for CICIDS 2017 and ESX-2, respectively. The t-SNE plots in the figure show that the normal and attack data points located nearby in the same space, which makes it very hard to classify them into either normal or attack. Although the t-SNE plots of normal and attack data are clustered, it clearly finds out that those are not linearly separated. In general it is known that deep learning is then effective at dealing with high-dimensional data with non-linearity [50], which is one of the reasons we employ deep learning approaches to detect cybert hreats.

Based on the results of this experiment, we are able to arrive at two meaningful conclusions. First, our mech-amiss are capable of being employed as learning- based models for network intrusion detection. When the performance evaluations were conducted using two well-known benchmark datasets such as NSLKDD and CICIDS2017, the result proved as capable as the conventional machine-learning models. This means that our proposed methods, employed in the AI-SIEM system, have applicability for learning-based network intrusion detection. Second, when the conventional learning-based methods, which accomplish good result by benchmark dataset, are employed in the real world, the performance of overall accuracy is not as reliable as those of benchmark datasets. Never the less, the accuracy performance of our three EP-ANN models were not significantly degraded, despite the large amount of data and a lack of benchmark data set features,such as seen in the result for ESX-2. By contrast, the accuracy of conventional methods had degraded from approximately 0.90 to0.85.

```
IPython console
Console 2/A
643/643 [==============================] - 0s 73us/step - loss: 0.3889 - acc: 0.8336
Epoch 87/100
643/643 [==============================] - 0s 73us/step - loss: 0.3868 - acc: 0.8367
Epoch 88/100
643/643 [==============================] - 0s 73us/step - loss: 0.3862 - acc: 0.8336
Epoch 89/100
643/643 [==============================] - 0s 97us/step - loss: 0.3851 - acc: 0.8383
Epoch 90/100
643/643 [==============================] - 0s 86us/step - loss: 0.3831 - acc: 0.8383
Epoch 91/100
643/643 [==============================] - 0s 99us/step - loss: 0.3806 - acc: 0.8383
Epoch 92/100
643/643 [==============================] - 0s 92us/step - loss: 0.3805 - acc: 0.8367
Epoch 93/100
643/643 [==============================] - 0s 90us/step - loss: 0.3801 - acc: 0.8351
Epoch 94/100
643/643 [==============================] - 0s 90us/step - loss: 0.3788 - acc: 0.8398
Epoch 95/100
643/643 [==============================] - 0s 93us/step - loss: 0.3772 - acc: 0.8398
Epoch 96/100
643/643 [==============================] - 0s 85us/step - loss: 0.3772 - acc: 0.8367
Epoch 97/100
643/643 [==============================] - 0s 73us/step - loss: 0.3743 - acc: 0.8414
Epoch 98/100
643/643 [==============================] - 0s 97us/step - loss: 0.3743 - acc: 0.8414
Epoch 99/100
643/643 [==============================] - 0s 97us/step - loss: 0.3729 - acc: 0.8398
Epoch 100/100
643/643 [==============================] - 0s 89us/step - loss: 0.3714 - acc: 0.8445
```

Epoch

In neural networks generally, an epoch is a single pass through a full dataset. It is the iterations constituting one forward pass and one backward pass. A confusion matrix is a technique for summarizing the performance of a classification algorithm. The number of correct and incorrect predictions are summarized with count values and broken down by each class. This is the key to the confusion matrix. Classification accuracy is the ratio of correct cyber threats. Computers do not generally store arbitrarily large numbers. Instead, each number stored by a computer is allotted a fixed amount of space. Therefore, when the number of time units that have elapsed since a system's epoch exceeds the largest number that can fit in the space allotted to the time representation, the time representation overflows, and problems can occur. While a system's behavior after overflow occurs is not necessarily predictable, in most systems the number representing the time will reset to zero, and the computer system will think that the current time is the epoch time again.

FUTURE WORK

With Intrusion Detection Systems and trained network security auditors, organizationshaveareliablemeanstoprioritizeandisolatethemostcriticalthreats in real time. They are tasked with identifying the significant elements of the attack and translating them into IDS signatures threat detection and response is the most important aspect of cyber security for IT organizations that depend on cloud infrastructure. Threat detection, therefore, describes the ability of IT organizations to quickly and accurately identify threats to the network or to applications or other assets within the network.

The proposed procedure changes large number of gathered security occasions over to singular occasion profiles and utilize a profound learning-based discovery strategy for upgraded digital danger identification. For this work, we built up an AI- SIEM framework dependent on a blend of occasion profiling for information preprocessing and distinctive counterfeit neural organization techniques, including FCNN, CNN, and LSTM. The framework centers around separating between obvious positive and bogus positive cautions, consequently causing security examiners to quickly react to digital dangers.

In this project, we have proposed the AI-SIEM system using event profiles and artificial neural networks .Then ovelty of our work lies in condensing very large-scale data into event profiles and using the deep learning-based detection methods for enhanced cyber-threat detection ability. The AI-SIEM system enables the security analysts to deal with significant security alerts promptly and efficiently by comparing long term security data. By reducing false positive alerts, it can also help the security analysts to rapidly respond to cyber threats dispersed across a large number of security events.

For the evaluation of performance, we performed a performance comparison using two bench mark data sets (NSLKDD,CICIDS2017)and two datasets collected in the real world. First, based on the comparison experiment with other methods, using widely known bench mark data sets, we showed that our mechanisms can be applied as one of the learning-based models for network intrusion detection. Second, through the evaluation using two real datasets, we presented promising results that our technology also outperformed conventional machine learning methods in terms of accurate classifications.

In the future, to address the evolving problem of cyber-attacks, we will focus on enhancing earlier threat predictions through the multiple deep learning approach to discovering the long-term patterns in history data. In addition, to improve the precision of labeled dataset for supervised-learning and construct good learning datasets, many SOC analysts will make efforts directly to record labels of raw security events one by one over several months. For testing, we constructed the purpose-built test bed where for conducting performance evaluations. This test bed consists of the big data platform and the AI-SIEM system. Moreover, in the SOC, we also had collected real-world IPS data over several months.

References

[1] S. Naseer, Y. Saleem, S. Khalid, M. K. Bashir, J. Han, M. M. Iqbal, K. Han, "Enhanced Network Anomaly Detection Based on Deep Neural Networks," *IEEE Access*, vol. 6, pp. 48231-48246,2018.

[2] B. Zhang, G. Hu, Z. Zhou, Y. Zhang, P. Qian, L. Chang, "Network Intrusion Detection Based on Directed Acyclic Graph and Belief Rule Base", *ETRI Journal*, vol. 39, no. 4, pp. 592-604, Aug.2017

[3] W. Wang, Y. Sheng and J. Wang, "HAST-IDS: Learning hierarchical spatial-temporalfeaturesusingdeepneuralnetworkstoimproveintrusiondetection,"*IEEE Access*, vol. 6, no. 99, pp. 1792-1806,2018.

[4] M. K. Hussein, N. Bin Zainal and A. N. Jaber, "Data security analysis for DDoS defense of cloud-based networks," *2015* IEEE Student Conference on Research and Development (Scored), Kuala Lumpur, 2015, pp.305-310.

[5] S. Sandeep Sekaran, K. Kandasamy, "Profiling SIEM tools and correlation enginesforsecurityanalytics," InProc.Int.Conf.WirelessCom.,SignalProve.and Net. (Wisp NET), 2017, pp.717-721.

[6] HubbellandV.SuryanarayanaFalsealarmminimizationtechniquesinsignature-based intrusion detection systems: A survey,"" Compute. Common., vol. 49, pp. 1-17, Aug.2014.

[7] A. Naser, M. A. Majid, M. F. Zolile and S. Anwar, "Trusting cloud computing for personal files," 2014 International Conference on Information and Communication Technology Convergence (ICTC), Busan, 2014, pp.488-489.

[8] Y. Shen, E. Marconi, P. Verviers, and Gianluca Stringham, "Tiresias: Predicting SecurityEventsThroughDeepLearning,"*InProc.ACMCCS18,*Toronto,Canada, 2018, pp.592-605.

[9] Kyle Soaks and Nicolas Christin, "Automatically detecting vulnerable websites beforetheyturnmalicious,",*InProc.USENIXSecuritySymposium.*,SanDiego,CA, USA, 2014,pp.625-640.

[10] K. Veerama channid, I. Arnaldo, V. Koraput, C. Basis, K. Li, "AI2: training abig datamachinetodefend,"InProc.IEEEBigdataSecurityHPSCIDS,NewYork,NY, USA, 2016, pp. 49-54

[11] Mahmood Lavallee, Ebrahim Bagheri, Wei Lu and Ali A. Ghobadi, "A detailed analysis of the kid cup 99 data set," In Proc. of the Second IEEE Int. Conf. Comp. Int. for Sec. and Def. App., pp.53-58, 2009.

[12] I.Sharfuddin,A.H.Lashari,A.A.Ghobadi,"Towardgeneratinganewintrusion detection dataset and intrusion traffic characterization", Proc. Int. Conf. Inf. Syst. Scur. Privacy, pp. 108- 116,2018.

[13] [online] Available:http://www.takakura.com/Kyoto_data/

[14] N. Shone, T. N. Ngoc, V. D. Phail and Q. Shi, "A deep learning approach to network intrusion detection," IEEE Trans. Emerge. Topics Compute. Intel., vol. 2, pp. 41-50, Feb.2018

[15] R.Vijayakumar,MamoonAlazar,K.P.Soman,P.PoornaChandran,AmeerAl-Namrata and Sit Lakshmi Venkatraman, "Deep Learning Approach for Intelligent Intrusion Detection System," IEEE Access, vol. 7, pp. 41525-41550, Apr.2019.

[16] W. Hu, W. Hu, S. Maybank, "Ad boost-based algorithm for network intrusion detection," IEEE Trans. Syst. Man B Cybernet., vol. 38, no. 2, pp. 577-583, Feb. 2008.

[17] T.-F. Yen et al., "Beehive: Large-scale log analysis for detecting suspicious activity in enterprise networks", Proc. 29th Annul. Compute. Security Appl. Conf., New York, NY, USA, 2013, pp. 199-208.

[18] K.-O. Darken, T Rix, C Kleiner, B Hellmann, L. Renner‟s, "Seem approach for a higher level of its security in enterprise networks", In Proc. IDAACS, Warsaw, Poland, 2015, pp.322-327.

[19] en.wikipedia.org “Security information and event management,” 2016 [Online] Available information and eventmanagement.

[20] Y. Lacuna, L. Bottom, Y. Bagnio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.

[21] C. Dong, C. C. Loy, K. He and X. Tang, "Image Super-Resolution Using Deep Convolutional Networks," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 38, no. 2, pp. 295- 307, 1 Feb.2016.

[22] A. Apathy, "Connecting images and natural language," Ph.D.dissertation, Fac. Compute. Sci., Stanford Univ., Stanford, CA, USA,2016.

[23] A.Krushinski,I.Subsieve,andG.E.Hinton,"ImageNetclassificationwithdeep convolutional neural networks," In Proc. Of the 25th Int. Confront Neural Inf. Proc. Systems -Volume 1, ser. NIPS‟12, 2012, pp.1097-1105.

[24] Q.Zhu,X.Li,A.Conesa,andC.Pereira,“Gram-CNN:adeeplearningapproach with local context for named entity recognition in biomedical text,” Bioinformatics, vol. 34, no. 9, pp. 1547–1554,2017.

[25] Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, “Malware traffic classification using convolutional neural network for representation learning,” In Proc. Int. Conf. on Infor. Net. (ICOIN), Da Nang, Vietnam, Jan. 2017, pp.712–717.

[26] Z. Li, Z. Qin, K. Huang, X. Yang, and S. Ye, “Intrusion detection using convolutionalneuralnetworksforrepresentationlearning,”InProc.Int.Conf.Neural Information Springer, 2017, pp.858–866.

[27] M. Alazar, S. Venkatraman, P. Watters, and M. Alazar, “Zero-day malware detection based on supervised learning algorithms of API call signatures,” In Proc. 9th Australis. Data Mining Conf., vol. 121. Ballarat, Australia, Dec. 2011, pp. 171- 182.

[28] E. Raff, J. Sylvester, and C. Nicholas, “Learning the PE header, malware detection with minimal domain knowledge," In Proc. 10th CalWORKs Atif. Intel. secure. New York, NY, USA, Nov. 2017, pp.121-132.

[29] J. Gu et al., "Recent advances in convolutional neural networks", Corer, pp. 187-332, Dec.2017

[30] KeneWu,ZuneChen,WeiLi,"ANovelIntrusionDetectionModelforaMassive Network Using Convolutional Neural Networks", Access IEEE, vol. 6, pp. 50850-50859,2018

[31] Taejon Kim, Sang C. Suh, Hyunjoo Kim, Jongmyo Kim and Jingo Kim, "An Encoding Technique for CNN-based Network Anomaly Detection," In Proc. IEEE InternationalConferenceonBigData(IEEEBigdata),Seattle,WA,USA,Jan.2019, pp.2960-2965.

[32] R. Vijayakumar, M. Alazar, K. P. Soman, P. Poorna Chandran and S. Venkatraman "Robust Intelligent Malware Detection Using Deep Learning," IEEE Access, vol. 7, pp. 46717-46738, Apr.2019

[33] Y. Lacuna, Y. Bagnio, and G. Hinton, “Deep learning,” Nature, vol. 521, no. 7553, pp. 436-444,2015.

[34] S. Hoch Reiter, Y. Bagnio, P. Francona, and J. Schmid Huber, “Gradient flow in recurrent nets: the difficulty of learning long-term dependencies,”2001.

[35] D. Similarities and Olivier Marko witch, "Reducing Data Complexity in Feature ExtractionandFeatureSelectionforBigDataSecurityAnalytics", InProc.Int.Conf. Data Intel. and Sec. (ICDIS), South Padre Island, TX, USA, May 2018, pp.43-48.

[36] V. N. Inoculum, S. Aris, S. R. Ravia, "Security issues associated with big data in cloud computing", International Journal of Network Security & Its Applications, vol. 6, no. 3, pp. 45,2014.